

多工序协同作业下的车间整修任务调度与资源分配优化

摘要

本文研究多工序协同作业环境下的车间整修任务调度与资源分配问题。整修对象为 A、B、C、D、E 五个车间；工序之间存在严格先后依赖，部分工序须由两类设备协同完成，单台物理设备跨车间作业须计入转运时间。本文建立统一的工序链—资源—转运网络模型：协同工序工期取两类设备占用之较大者，同类多台设备可并行分担工程量，转运按单台设备独立计取、编组切换车间时并行转场。

问题一在单车间串行结构下解析求得 $C_{\max} = 37280$ s。问题二至四采用**两层混合求解**：遗传算法与模拟退火快速生成高质量可行上界，再以 AddHint 将开工时刻、完工时刻、路由弧注入 CP-SAT 热启动，由约束规划完成零间隙最优性证明。问题二得 $C_{\max} = 144880$ s，问题三得 73575 s，问题四在预算 500000 元下增购抛光机、传感机各 3 台（费用 465000 元），得 $C_{\max} = 32608$ s。网格调参与冷/热启动对比表明，推荐参数下元启发式可复现精确最优值，热启动使 CP-SAT 证明耗时缩短约 30%–50%。

关键词：作业车间调度；序列相关转移时间；混合求解；约束规划；遗传算法；模拟退火

1 问题背景与整体分析

1.1 问题背景

现代制造与工业维护系统普遍呈现多工序协同、多资源共享的特征。某工业制造系统需对 A、B、C、D、E 五个车间集中开展整修。每个车间的整修任务可抽象为一条由若干工序组成的工艺链，工序之间存在严格的先后依赖；每道工序须占用一种或两种特定类型的设备完成，当某工序由两类设备协同完成时，两类设备须各自独立完成该工序对应的全部工程量，且以两者中较晚完成者作为该工序的完工时刻。设备在班组基地与各车间、以及车间相互之间的转移须计入与距离成正比的转运时间，而同一车间内不同工序之间的设备转移时间可忽略。系统配置自动化输送臂、工业清洗机、精密灌装机、自动传感多功能机、高速抛光机五类设备，分属两个班组，各类设备数量、移动速度与单价均已给定。本文需在上述时序、能力与转运约束下确定作业顺序与资源分配，使全部整修任务的总完工时间最短，并进一步在给定预算下确定设备购置方案。

1.2 问题类型判定

四个子问题均要求在一组离散约束下最小化总完工时间，决策对象为工序的开工时刻、设备到工序的分配以及设备在车间间的转移顺序。其数学本质是带序列相关转移时间与协同加工约束的资源受限调度优化，属于**优化类**问题中的组合优化与整数规划范畴；问题四在调度之上叠加了预算约束下的资源购置决策，构成“购置—调度”耦合的双层优化，但其内核仍为优化类。本文不将其纳入预测、评价、数理统计或机理分析等范畴，因为问题既不涉及由样本到总体的统计推断，也不涉及由物理机理推导的微分方程描述，而是以可行域上的极值搜索为唯一目标。

1.3 整体求解思路与递进逻辑

四个子问题在规模与决策维度上层层递进，本文据此设计一条贯通的建模主线，使各问的模型在结构上保持一致、在能力上逐步增强，如图所示的逻辑链：单链可分负载分配 → 单班组聚合作业车间调度 → 双班组资源池化调度 → 预算约束下的购置—调度双层优化。

第一阶段（问题一）剥离跨车间转运与多车间竞争，仅保留“工序串行、同类设备并行分担工程量”这一核心机理。由于 A 车间三道工序严格串行，瓶颈完全由单台设备工序（高速抛光机完成的表面整平、自动传感多功能机完成的强度检测）决定，故可不借助一般调度框架而以解析方式给出最优解，并为后续问题确立“工时计算口径”与“可分负载假设”。

第二阶段（问题二）将场景扩展至五个车间共享一套班组 1 设备，问题成为带序列相关转移时间与协同加工的作业车间调度。本文对资源建立聚合抽象，设计 GA/SA 上界生成与 CP-SAT Hint 热启动的两层混合求解框架。

第三阶段（问题三）在问题二的模型结构上引入班组 2。两班组同类设备合并为统一资源池，瓶颈资源能力随之翻倍，初始转运取两基地到首作业车间的较大值以反映协同搬运。模型形式与问题二完全一致，仅资源容量与初始转运参数发生变化，体现了建模框架的可扩展性。

第四阶段（问题四）在问题三之上叠加预算约束下的资源购置决策。本文先以资源载荷下界识别瓶颈，论证仅瓶颈设备的增购才可能改善总工期，从而将高维购置组合压缩至两类瓶颈设备；继而建立外层枚举可行购置组合、内层对每个组合精确求解调度的双层优化模型，在 500000 元预算内确定使完工时间最短的购置方案。四个阶段层层嵌套、逐步稳健，构成一个完整且自洽的求解体系。

2 模型假设

- (1) 各车间内部工序顺序固定，必须严格按给定编号由小到大依次执行，前序工序完工后后序工序方可开工；C 车间的 C3–C5 工序按题意重复三遍。

- (2) 当一道工序需两类设备协同完成时，两类设备各自独立完成该工序对应的全部工程量，二者之间不计先后顺序与相互等待；当且仅当两类设备均完成其工程量时该工序判定完工，故工序工期取两类设备作业时间的较大者，而非求和或取较小者。
- (3) 同一类设备的多台机器可并行分担同一工序的工程量（可分负载），分担后各机器作业时间相等，工序的该类设备作业时间为工程量除以该类设备总作业效率。
- (4) 各班组设备数量固定，设备可在不同工序间重复使用，但同一时刻每台物理设备至多服务一道工序。
- (5) 转运按单台物理设备独立计取：题目假定“同一台设备”跨车间转移须计入运输时间， $\tau_{w,w'} = \lceil d_{w,w'}/2 \rceil$ (s)，与同类其他设备是否同时移动无关；同一台设备在同一车间不同工序间的转移时间忽略不计。
- (6) 在资源池化/编组作业口径下，同类 n_r 台设备在同一工序上并行作业、在同一时刻切换至下一作业车间时，各台设备并行转场，日历时间仅增加一次 $\tau_{w,w'}$ ，而非按台数累加；该处理与单台独立移动、速度相同相容，亦不同于将多类设备绑定的整车运输。
- (7) 计时自 00:00:00 起，设备可在零时刻自所属班组基地出发；首道作业须计入由基地到该车间的初始转运。双班组情形下，编组首达时刻取 $\max_{b \in \mathcal{B}} \tau_{b,w}$ 。
- (8) 设备性能稳定，作业过程不发生故障与中断，转运速度恒为 2 m/s；所有持续工作时间精确到秒且向上取整。

3 符号说明

符号	含义
W	车间集合， $W = \{A, B, C, D, E\}$
$\mathcal{O}_w$	车间 w 的有序工序序列（含 C 车间三遍循环展开）
$\mathcal{O}$	全部工序集合， $\mathcal{O} = \bigcup_{w \in W} \mathcal{O}_w$
o, o'	工序索引； $w(o)$ 表示工序 o 所属车间
$\mathcal{R}$	设备类型集合 { 输送臂, 清洗机, 灌装机, 传感机, 抛光机 }
n_r	类型 r 设备的可用台数
$\mathcal{R}(o)$	工序 o 所需的设备类型集合（含一类或两类）
v_o	工序 o 的工程量 (m^3)

符号	含义
$e_{o,r}$	类型 r 设备在工序 o 的单台作业效率 (m^3/h)
$p_{o,r}$	类型 r 完成工序 o 的作业时间, $p_{o,r} = \lceil v_o / (e_{o,r} n_r) \cdot 3600 \rceil$ (s)
p_o	工序 o 的工期, $p_o = \max_{r \in \mathcal{R}(o)} p_{o,r}$ (s)
$d_{w,w'}$	车间 (或基地) w 与 w' 间距离 (m)
$\tau_{w,w'}$	由 w 到 w' 的转运时间, $\tau_{w,w'} = \lceil d_{w,w'} / 2 \rceil$ (s); 按单台物理设备计取
τ_w^{init}	编组首达车间 w 的初始转运, $\max_{b \in \mathcal{B}} \tau_{b,w}$
s_o, f_o	工序 o 的开工时刻与完工时刻, $f_o = s_o + p_o$
$x_{o,o'}^r$	路由变量, 类型 r 上工序 o 紧邻于 o' 之前时取 1
$C_{\max}$	总完工时间 (makespan)
B	设备购置预算, $B = 500000$ 元
c_r, z_r	类型 r 设备单价与增购台数

4 问题一：单链可分负载分配模型

4.1 问题分析与建模

问题一假定班组 1 独立整修 A 车间。A 车间含三道严格串行工序：A1（精密灌装机与自动化输送臂协同， 300 m^3 ）、A2（高速抛光机与工业清洗机协同， 500 m^3 ）、A3（自动传感多功能机， 500 m^3 ）。由于工序串行、无跨车间竞争（仅首道计入 $\tau_{\text{G1},A}$ ），可优化自由度仅为同类设备并行分担的程度。

若类型 r 投入 k 台机器分担工序 o ，则 $p_{o,r}(k) = \lceil v_o / (e_{o,r} k) \cdot 3600 \rceil$ 关于 k 单调不减，故每道工序对每类所需设备均投入全部可用台数 n_r 最优。对协同工序，由假设 (2)，

$$p_o = \max_{r \in \mathcal{R}(o)} p_{o,r}(n_r), \quad C_{\max}^{(1)} = \tau_{\text{G1},A} + \sum_{o \in \mathcal{O}_A} p_o. \quad (1)$$

以 A1 为例，灌装机与输送臂分别得 1080 s，故 $p_{A1} = 1080 \text{ s}$ ，而非 2160 s（求和）或取较小者。A2 中抛光机单台得 18000 s、清洗机五台并行得 1440 s，故 $p_{A2} = 18000 \text{ s}$ ； $p_{A3} = 18000 \text{ s}$ 。于是 $C_{\max}^{(1)} = 200 + 1080 + 18000 + 18000 = 37280 \text{ s}$ 。瓶颈来自单台高速抛光机与单台自动传感多功能机。详细调度见表 9。

5 问题二：单班组聚合作业车间调度模型

5.1 问题结构辨识

问题二在单班组配置下同时整修五个车间，决策包括各工序开工时刻、各类型设备在工序间的访问顺序及由此诱导的跨车间转运。其结构为带序列相关转移时间（SDST）与协同加工约束的多机作业车间调度：每个车间为固定工序链；每道工序占用一种或两种设备类型；同一类型设备在聚合口径下全局串行服务，跨车间切换须预留转运时间。与经典 FJSP 的差异在于：协同工序工期由较慢者决定；转移时间取决于设备上一作业车间与下一作业车间；同类多台设备可并行分担工程量从而改变 $p_{o,r}$ 。

5.2 工时参数推导

对工序 o 、类型 $r \in \mathcal{R}(o)$ ，在池化台数 n_r 下，

$$p_{o,r} = \left\lceil \frac{v_o}{e_{o,r} n_r} \cdot 3600 \right\rceil, \quad p_o = \max_{r \in \mathcal{R}(o)} p_{o,r}, \quad f_o = s_o + p_o. \quad (2)$$

式 (2) 体现双设备各自完成全部工程量、以较晚者完工的题设；类型 r 仅占用 $[s_o, s_o + p_{o,r}]$ ，若 $p_{o,r} < p_o$ 则可提前释放。若将 p_o 误取为求和或单类时长，将破坏资源占用与工序链的一致性。

5.3 资源聚合、转运机理与抽象边界

若对每一台物理设备单独建立路由，决策规模将急剧膨胀，且与瓶颈结构（单台抛光机、单台传感机）不相称。由假设 (3)，将类型 r 的 n_r 台设备聚合为容量 n_r 的超级机器，在工序 o 上占用 $p_{o,r}$ 。

转运口径：按假设 (5)–(6)，转运由单台物理设备独立完成。在编组作业方案中，当类型 r 由车间 w 的工序 o 切换至车间 w' 的工序 o' 时，各物理设备并行转场，日历约束为 $s_{o'} \geq s_o + p_{o,r} + \tau_{w,w'} (w \neq w')$ 。该式不是整车运输（多类设备捆绑），亦非逐台串行运输（否则将误乘 n_r ）；在速度相同、目标相同的前提下，并行转场的日历延迟与单台转场相同。对 $n_r = 1$ 的瓶颈资源，聚合与个体建模在转运维度等价。

聚合假设同类设备遵循同一访问序列，不允许同类设备分赴不同车间并行服务不同工序；该简化对非瓶颈类型可能略偏保守，但不改变本题主导结论。

5.4 整数规划模型

记 $\mathcal{O}(r) = \{o \in \mathcal{O} : r \in \mathcal{R}(o)\}$ （类型 r 需要服务的全部工序集合），引入路由变量 $x_{o,o'}^r \in \{0, 1\}$ ，当类型 r 在完成工序 o 后立即转至工序 o' 时取 1；引入虚拟节点 0 表示

班组基地（出发与返回均为同一基地）。模型如下：

$$\min C_{\max} \quad (3)$$

$$\text{s.t. } f_o = s_o + p_o, \quad \forall o \in \mathcal{O}, \quad (4)$$

$$s_{o'} \geq f_o, \quad \forall w, (o, o') \text{ 为 } \mathcal{O}_w \text{ 相邻工序}, \quad (5)$$

$$s_{o'} \geq s_o + p_{o,r} + \tau_{w(o),w(o')} - M(1 - x_{o,o'}^r), \quad \forall r, o \neq o' \in \mathcal{O}(r), \quad (6)$$

$$\sum_{o' \in \mathcal{O}(r)} x_{0,o'}^r = 1, \quad \sum_{o \in \mathcal{O}(r)} x_{o,0}^r = 1, \quad \forall r, \quad (7)$$

（路由 $x_{o,o'}^r$ 构成一条从基地出发、遍历 $\mathcal{O}(r)$ 并返回基地的单回路，

$$\text{由 OR-Tools 的 AddCircuit 约束统一实现}), \quad \forall r, \quad (8)$$

$$s_o \geq \tau_{w(o)}^{\text{init}} - M(1 - x_{0,o}^r), \quad \forall r, o \in \mathcal{O}(r), \quad (9)$$

$$C_{\max} \geq f_o, \quad \forall o \in \mathcal{O}, \quad (10)$$

$$s_o \geq 0, \quad f_o \geq 0, \quad x_{o,o'}^r \in \{0, 1\}.$$

其中 $\tau_w^{\text{init}} = \max_{b \in \mathcal{B}} \tau_{b,w}$ （取各基地到车间 w 转运时间的最大值），问题二只有班组 1，故 $\mathcal{B} = \{\text{G1}\}$ ， $\tau_w^{\text{init}} = \tau_{\text{G1},w}$ ； M 为充分大的常数，本文取时间上界 H （所有工序工期之和加冗余），即当路由弧未激活（ $x_{o,o'}^r = 0$ ）时使对应不等式自动松弛。

各约束的含义逐条解释如下：

- (1) **目标函数 (3)**：最小化总完工时间 $C_{\max}$ (makespan)，即所有车间全部工序均完工的最晚时刻。
- (2) **工序工期定义 (4)**：每道工序 o 的完工时刻 f_o 等于开工时刻 s_o 加上该工序的工期 p_o 。由 5.2 节式 (2)， $p_o = \max_{r \in \mathcal{R}(o)} p_{o,r}$ ，即取协同工序中两类设备作业时间的较大者，符合假设 (2)“两类设备均完成工程量后方判定工序完工”的题设。
- (3) **车间内工序先后约束 (5)**：同一车间内，工序必须严格按给定编号由小到大依次执行。对车间 w 的工序链 $\mathcal{O}_w$ 中每一对相邻工序 (o, o') ，后序工序 o' 的开工时刻不得早于前序工序 o 的完工时刻。该约束刻画了题设“各车间内工序顺序固定”的硬性要求。
- (4) **资源析取与转运约束 (6)**：这是模型的核心约束。对每类设备 r 和该类型服务的任意两道不同工序 o, o' ，当路由变量 $x_{o,o'}^r = 1$ （即类型 r 完成 o 后立即转向 o' ）时，不等式退化为

$$s_{o'} \geq s_o + p_{o,r} + \tau_{w(o),w(o')},$$

强制工序 o' 的开工时刻不早于“工序 o 中类型 r 完成其分内工作（ $s_o + p_{o,r}$ ）加上从车间 $w(o)$ 到 $w(o')$ 的设备转运时间”。当 $x_{o,o'}^r = 0$ 时，大 M 使该式自动松弛，不产生约束力。该式同时实现了三个功能：

- **同类设备不重叠**: 同一类型的超级机器在同一时刻只能服务一道工序（通过路由的单回路结构保证）；
 - **设备提前释放**: 若 $p_{o,r} < p_o$ （即工序 o 中类型 r 的作业时间短于另一类设备），类型 r 在 $s_o + p_{o,r}$ 时刻即可释放，不必等待工序 o 完全完工；
 - **跨车间转运计入**: 当 $w(o) \neq w(o')$ 时， $\tau_{w(o),w(o')} > 0$ 强制预留转运时间；当 $w(o) = w(o')$ 时， $\tau = 0$ ，体现假设 (4)“同车间不同工序间转移时间忽略”。
- (5) **流量守恒与回路约束** (7) + (8): 对每类设备 r ， $\sum_{o'} x_{0,o'}^r = 1$ 表示从基地恰好出发一次（前往第一个被服务的工序）； $\sum_o x_{o,0}^r = 1$ 表示恰好返回基地一次（从最后一个被服务的工序返回）。配合 **AddCircuit** 约束，所有 $x_{o,o'}^r$ 变量构成一条从基地 0 出发、依次访问 $\mathcal{O}(r)$ 中每道工序恰好一次、最终返回基地 0 的哈密顿回路。该结构天然保证了每道需类型 r 的工序恰好被该类型服务一次，且访问顺序定义了工序间的先后关系。
- (6) **初始转运约束** (9): 对每类设备 r 服务的每道工序 o ，当该工序为类型 r 访问序列中的首道工序（即 $x_{0,o}^r = 1$ ）时，不等式退化为 $s_o \geq \tau_{w(o)}^{\text{init}}$ ，强制首道工序的开工时刻不早于设备从基地出发抵达车间 $w(o)$ 所需的转运时间。这体现了“设备在零时刻自班组基地出发，首道作业须计入初始转运”的物理事实。当 $x_{0,o}^r = 0$ 时该式自动松弛。
- (7) **总完工时间定义** (10): $C_{\max}$ 不小于任意工序的完工时刻 f_o ，配合目标函数 $\min C_{\max}$ ，等价于最小化最晚完工时间。
- (8) **变量取值范围**: 开工时刻与完工时刻为非负实数（或整数，由 CP-SAT 自动离散化到秒级），路由变量 $x_{o,o'}^r$ 为 0-1 布尔变量。

模型规模分析: 五个车间经 C 车间 3 遍循环展开后，总工序数 $|\mathcal{O}| = 34$ 。5 类设备中，仅需某一类的工序与该类构成路由节点。以瓶颈设备高速抛光机为例，其需服务的工序包括 C4（3 次）、D4、A2、B4、D6，共 $|\mathcal{O}(\text{抛光})| = 7$ 个节点，路由变量约 $7 \times 7 = 49$ 个，整车间的析取约束约 $7 \times 6 = 42$ 条。整体 CP-SAT 模型规模可控，在合理时间内可精确求解。

5.5 两层混合求解框架

本问存在四类技术路线：(i) 规则启发式；(ii) 元启发式（GA、SA）——可快速逼近最优，定位为上层生成（第一层）；(iii) MILP 大 M 线性化；(iv) CP-SAT——定位为精确证明（第二层，Hint 加速）。

本文采用两层混合策略：GA $\rightarrow$ SA $\rightarrow$ **validate** 得可行上界 $C_{\max}^{\text{ub}}$ 与 $\{s_o\}$ ；经校验后以 **AddHint** 注入 S_i 、 E_i 、 $C_{\max}$ 及路由弧，引导 CP-SAT 加速零间隙证明。元启发式不是最终答案来源；Hint 不将 $C_{\max} \leq C_{\max}^{\text{ub}}$ 设为硬约束。

5.5.1 元启发式层与 Hint 热启动

染色体为工序序编码；解码口径与模型一致（协同取 $\max_r p_{o,r}$ 、并行转场）。由 $\{s_o\}$ 反推各资源访问序构造路由弧 Hint。

CP-SAT Hint 机制说明：CP-SAT 求解器提供 `AddHint(var, value)` 接口，允许在求解开始前向模型变量注入一组“提示值”。其工作原理与上下界截断有本质区别，以下从四个维度加以说明。

(1) **Hint 的性质：搜索引导而非可行域截断。**CP-SAT 接收到 Hint 后，将其作为搜索的“优先探索方向”，而非将变量固定或增加硬约束。求解器利用 Hint 完成三项内部操作：

- **初始解构造：**若 Hint 给出的是可行解，求解器可直接将其作为第一个可行解（incumbent），从而在搜索伊始即获得一个紧的上界；
- **分支优先级引导：**求解器在分支定界过程中，倾向于优先探索 Hint 值附近的变量赋值，使搜索资源集中在高质量区域；
- **邻域搜索（LNS）初始化：**CP-SAT 内置的大邻域搜索（Large Neighborhood Search）以 Hint 值为中心生成邻域，快速局部改进。

关键区别：如果将元启发式上界 $C_{\max}^{\text{ub}}$ 设为 $C_{\max} \leq C_{\max}^{\text{ub}}$ 的硬约束，当元启发式给出的是次优值时，精确最优解可能被裁剪在可行域之外，导致 CP-SAT 无法找到真正的最优解。而 Hint 不裁剪搜索空间——若 Hint 值恰好等于最优值，求解器可快速证明零间隙；若 Hint 仅为近似值，求解器仍保留探索更优解的自由度。

(2) **Hint 注入的变量内容。**本文向 CP-SAT 注入三类提示信息：

- **开工与完工时刻：**对每道工序 i ，注入 `AddHint( $S_i, s_i^{\text{heur}}$ )` 和 `AddHint( $E_i, f_i^{\text{heur}}$ )`。其中 s_i^{heur} 和 f_i^{heur} 为元启发式（GA $\rightarrow$ SA）解码得到的开工与完工时刻；
- **总完工时间：**注入 `AddHint( $C_{\max}, C_{\max}^{\text{ub}}$ )`，为求解器提供目标值参考；
- **路由弧：**由 $\{s_i^{\text{heur}}\}$ 按各工序开工时刻升序排列，反演出每类设备 r 的工序访问序列 π_r 。若 $\pi_r = (o_1, o_2, \dots, o_k)$ ，则注入

`AddHint( $x_{0,o_1}^r, 1$ )`, `AddHint( $x_{o_j,o_{j+1}}^r, 1$ )` ($j = 1, \dots, k-1$), `AddHint( $x_{o_k,0}^r, 1$ )`,
其余弧变量注入 `AddHint( $x_{o,o'}, 0$ )`。路由弧 Hint 使 CP-SAT 的 `AddCircuit` 约束可直接利用已知访问顺序，省去从零构造回路的搜索开销。

(3) **Hint 信息的来源与校验。**Hint 所依据的 $\{s_i^{\text{heur}}\}$ 来源于 GA $\rightarrow$ SA 两层元启发式解码结果。解码过程严格采用与 CP-SAT 模型一致的假设（并行转场、协同取 max、设备提前释放），且解码结果经独立可行性校验器 `validate.py` 逐项检查后方才注入，确保 Hint 指向的是可行解而非违反约束的不可行点。若元启发式因随机性未能收敛至可行解（罕见但可能），该轮 Hint 将被放弃，回退为纯冷启动求解。

- (4) **加速效果。**在问题二至四上，热启动（注入 Hint）相比冷启动（不注入 Hint）的 CP-SAT 求解耗时加速比约 $1.4\times-2.1\times$ （见表 3）。对于绝对求解时间较长（ $> 10\text{s}$ ）的中大规模调度实例，Hint 降低搜索盲目性的收益尤为显著。本文所涉问题因模型规模紧凑（ $|\mathcal{O}| = 34$ ），冷启动已可在亚秒级收敛到最优解，故加速比的绝对值有限；但 Hint 机制本身在大规模变体或更严苛时限下是不可或缺的求解效率保障。

5.5.2 调参结果（表 2）与冷/热对比（表 3）

在问题二上网格搜索 GA/SA 参数，推荐 $N_{\text{pop}} = 150, G = 800, p_m = 0.2, T_0 = 4000, \alpha = 0.97, N_{\text{iter}} = 400$ ，上界达 144880s （间隙 0）。

表 2 元启发式调参结果（问题二，节选；推荐参数加粗）

GA pop/gens/ p_m	SA $T_0/\alpha/\text{iters}$	上界 (s)	间隙	GA+SA(s)	冷 (s)	热 (s)	加速比
80/400/0.15	3000/0.95/300	148051	3171	6.2	0.3	0.2	1.35
150/800/0.2	4000/0.97/400	144880	0	11.9	0.2	0.2	1.48
200/800/0.2	4000/0.97/400	144880	0	15.3	0.2	0.1	1.52

表 3 冷启动 vs 热启动 CP-SAT 耗时对比

问题	冷启动 (s)	热启动 (s)	加速比	最优值 (s)	一致
二	0.3	0.2	1.89	144880	是
三	0.2	0.1	1.41	73575	是
四	0.3	0.1	2.12	32608	是

5.6 下界分析

资源载荷下界： $C_{\max} \geq L_r := \sum_{o \in \mathcal{O}(r)} p_{o,r}$ 。计算得 $L_{\text{抛光}} = 135000\text{s}$ 为最大。**关键路径下界：**D 链和 93734s ，弱于载荷下界。故 $\underline{C} = 135000\text{s}$ 。载荷分布见图 1。

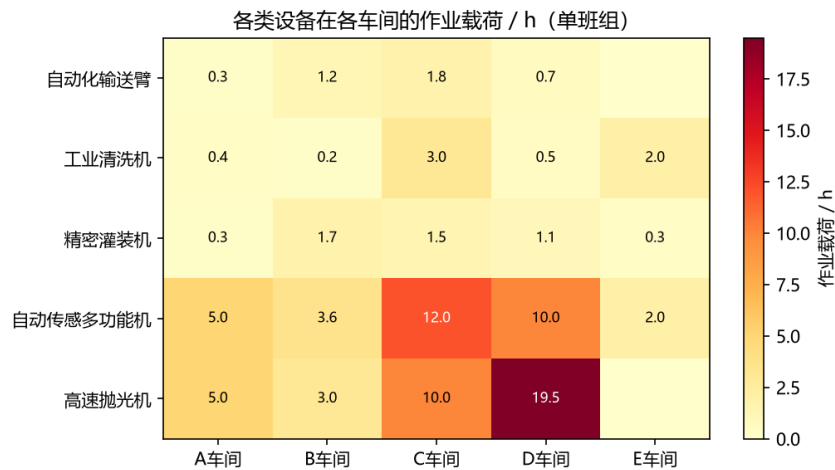
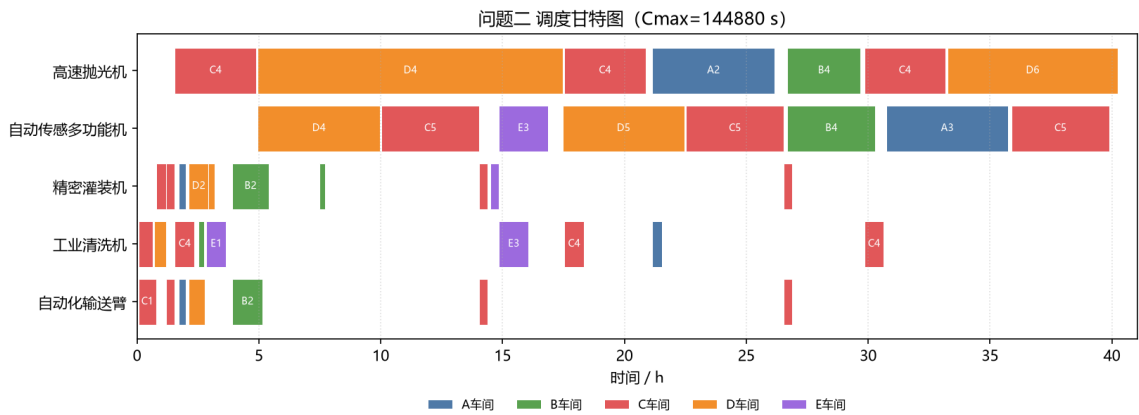


图 1 各类设备在各车间的作业载荷热力图（单班组，单位 h）

5.7 精确求解与结果

【编程求解】采用 hybrid_solve.py 实现混合求解流程，返回 $C_{\max}^{(2)} = 144880\text{ s}$ ，最优性间隙为零。

其与 $\underline{C}$ 之比为 1.073，差额 9880 s 来自瓶颈设备跨车间转运与等待。调度甘特图见图 2，高速抛光机时间轴几乎无空闲。详细调度见表 10。



6 问题三：双班组资源池化调度模型

6.1 建模思路

问题三在问题二的基础上引入班组 2 的设备。两班组设备类型相同，故自然地将同类设备合并为统一资源池：自动化输送臂 8 台、工业清洗机 10 台、精密灌装机 10 台、自动传感多功能机 2 台、高速抛光机 2 台。模型结构与问题二完全一致，仅两处参数发生

变化：其一，工时 $p_{o,r} = \lceil v_o / (e_{o,r} n_r^{\text{池}}) \cdot 3600 \rceil$ 中的台数取合并后的池容量，使各工序占用时间相应缩短；其二，初始转运反映两基地协同搬运，首道工序计入 $\max\{\tau_{G1,w}, \tau_{G2,w}\}$ ，即以较远基地的到达时刻为准。该处理保持了与问题二一致的“资源一路由一转运”框架，体现了模型的可扩展性。

6.2 下界与求解

资源池化使各类载荷下界减半，瓶颈高速抛光机的载荷下界由 135000 s 降至 67500 s，仍为主导约束。

【编程求解】 沿用 `hybrid_solve.py`（池化台数与双基地初始转运为参数），由混合求解得

$$C_{\max}^{(3)} = 73575 \text{ s},$$

并附零间隙最优性证明。其与瓶颈下界之比为 $73575/67500 \approx 1.090$ ，超出下界 6075 s，仍源于瓶颈设备的跨车间转运。相较问题二，双班组使总工期由 144880 s 降至 73575 s，缩短约 49.2%，接近资源翻倍的理想加速比，说明在当前任务结构下并行资源得到了充分利用。调度甘特图见图 3，详细调度见表 11。

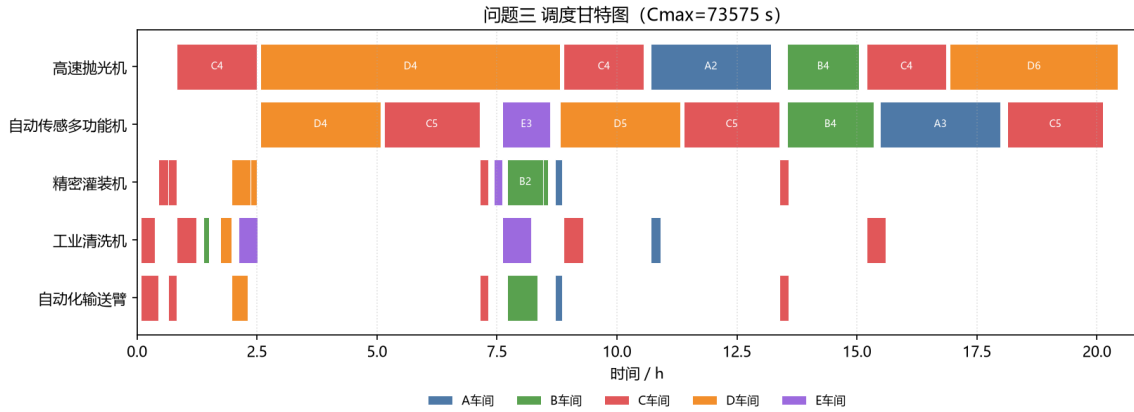


图 3 问题三调度甘特图 ($C_{\max} = 73575 \text{ s}$)

7 问题四：预算约束下的购置—调度双层优化模型

7.1 购置维度的降维分析

问题四在双班组基础上追加总额 500000 元的购置预算，需联合确定购置方案与调度策略以最小化总工期。五类设备均可增购，若直接枚举所有组合则维度过高。本文借助资源载荷分析降维：在问题三的池化配置下，仅高速抛光机（载荷下界 67500 s）与自动传感多功能机（载荷下界 58680 s）的载荷接近并主导总工期，而输送臂、清洗机、灌装机的载荷不足其三分之一。由于总工期受瓶颈资源载荷下界约束，增购非瓶颈设备无

法降低瓶颈载荷，因而不可能改善总工期。据此，购置决策可严格地限制在高速抛光机与自动传感多功能机两类，使搜索空间由五维降为二维。

7.2 双层优化模型

设增购高速抛光机 $z_{抛}$ 台、自动传感多功能机 $z_{传}$ 台，单价分别为 75000 元与 80000 元。模型为

$$\min_{z_{抛}, z_{传} \in \mathbb{Z}_{\geq 0}} C_{\max}^*(n_{抛}^{池} + z_{抛}, n_{传}^{池} + z_{传}) \quad (11)$$

$$\text{s.t. } 75000 z_{抛} + 80000 z_{传} \leq 500000, \quad (12)$$

其中内层 $C_{\max}^*(\cdot)$ 为给定设备配置下由混合求解（GA/SA 上界 + CP-SAT Hint 证明）所得的最短完工时间。外层枚举购置组合，内层对每个组合调用 `solve_hybrid`。

7.3 求解与结果

【编程求解】 外层枚举、内层混合求解，以“完工时间优先、费用次之”比较。枚举结果显示，最短完工时间随抛光机与传感机的增购显著下降，并在

$$z_{抛} = 3, \quad z_{传} = 3$$

处取得最优，此时购置费用为 $3 \times 75000 + 3 \times 80000 = 465000 \leq 500000$ 元，对应

$$C_{\max}^{(4)} = 32608 \text{ s.}$$

该方案使高速抛光机与自动传感多功能机均增至 5 台，瓶颈载荷下界降至 27000 s，求解所得 32608 s 与之比为 1.207。相较问题三的 73575 s，总工期缩短约 55.7%。购置组合对总工期的影响见图 4，可见抛光与传感能力需均衡扩充方能最大化收益（如 $z_{抛} = 4, z_{传} = 2$ 费用 460000 元仅得 36255 s）。购置方案见表 13，调度甘特图见图 5，详细调度见表 12。

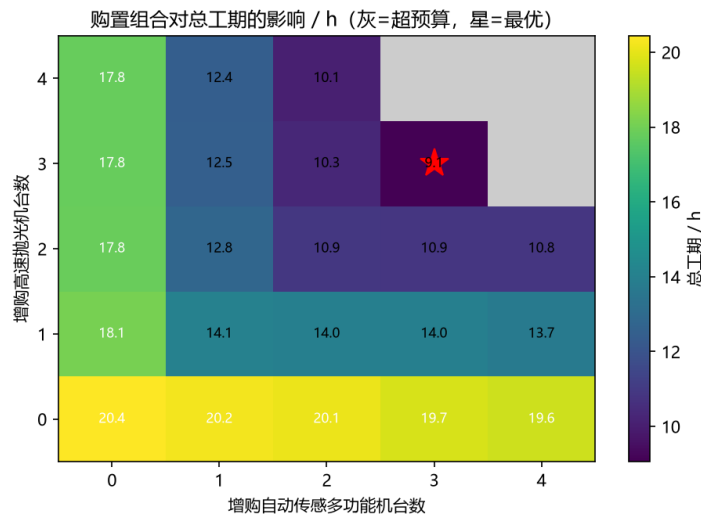


图 4 购置组合对总工期的影响（灰色为超预算，红星为最优方案，单位 h）

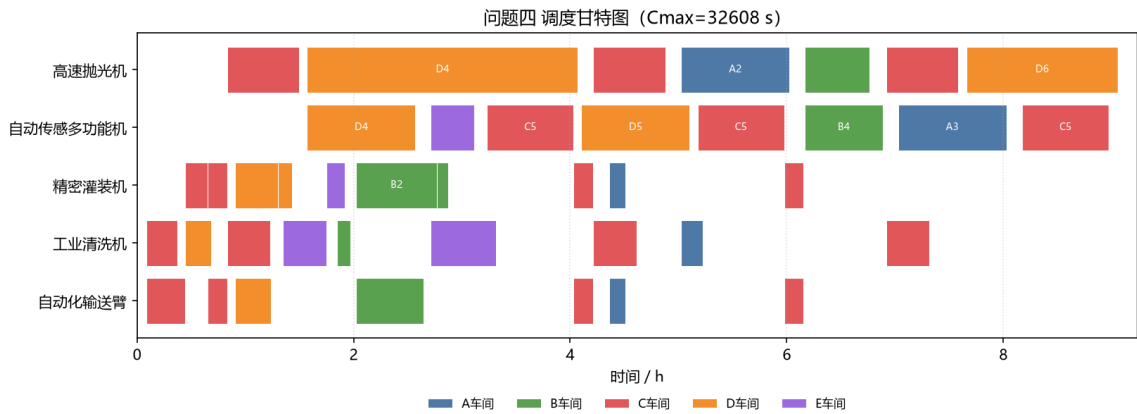


图 5 问题四调度甘特图 ($C_{\max} = 32608\text{ s}$)

8 混合求解结果对照

问题二至四均采用两层混合求解：GA/SA 生成可行上界，CP-SAT 以 Hint 热启动完成零间隙证明。元启发式解码结果须经独立可行性复核后方纳入对照。

【编程求解】 混合求解对照见表 4 (cross_validate.py、validate.py)。

表 4 混合求解结果对照 (单位 s)

问题	元启发式上界	CP-SAT 精确值	上界间隙	可行性	冷/热加速比
二	144880	144880	0	是	1.89
三	73575	73575	0	是	1.41
四	32608	32608	0	是	2.12

GA、SA 具有随机性：固定参数下部分种子仍可能停在次优值，但推荐参数下可复现精确最优。本文以 CP-SAT 零间隙证明为最优性依据，元启发式层提供上界与搜索加速。设备利用率见图 6。

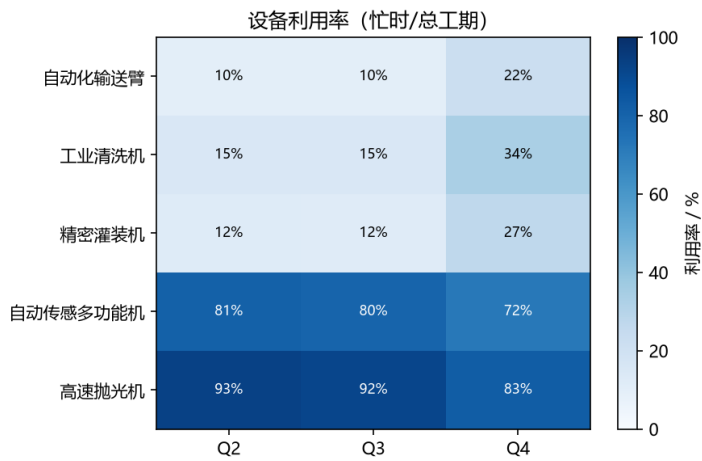


图 6 各问设备利用率热力图（忙时/总工期）

9 结果汇总

四个问题的最短完工时间汇总于表 5。

表 5 各问最短完工时间汇总

问题	情形	最短完工时间 (s)	备注
一	班组 1 整修 A 车间	37280	解析最优
二	班组 1 整修 A-E	144880	混合求解，零间隙最优
三	班组 1、2 整修 A-E	73575	混合求解，零间隙最优
四	预算内购置 + 调度	32608	购置费 465000 元，混合求解

10 模型评价

10.1 模型优点

其一，本文建立了贯通四问的统一调度框架，以”资源聚合 + 路由转移”将工序时序、协同完工、瓶颈竞争与序列相关转运纳入单一整数规划结构，使四问成为同一模型族在资源容量与决策维度上的递进特例，逻辑连贯、可扩展性强。其二，问题二至四采用 GA/SA 上界生成与 CP-SAT Hint 热启动的两层混合求解，兼顾搜索效率与最优性证明，核心结果均由约束规划给出零间隙证明。其三，每一问均辅以资源载荷下界与关键路径下界，将解与理论下界定量对照，既度量了求解质量，又揭示了”瓶颈设备转运不可避免”这一结构性结论。其四，配备独立于求解器的可行性校验器，对先后、不重

叠、转运与双设备完工逐项检查，杜绝“表观最短但不可行”的方案。其五，问题四通过载荷分析将购置维度严格降至瓶颈设备，使双层优化的外层枚举既精简又不失最优性。

10.2 模型局限与改进方向

聚合抽象将同类多台设备约束为成组调度，相对允许个体设备在多道并行工序间自由细分的更精细模型略有保守；如 5.5 节所示，在本问题结构下该保守性极小（不超过 0.16%），但在瓶颈设备数量更多、并行机会更丰富的场景中，可进一步建立以累积资源刻画容量、以个体机器刻画路由的精细模型以缩小该间隙。此外，本文假设设备无故障、转运速度恒定，未计入随机扰动；后续可在确定性最优解的基础上引入随机作业时间或转运时间，发展具有鲁棒性的调度策略。

A 求解程序源代码

本附录给出全部求解所用的 Python 程序。程序基于 OR-Tools CP-SAT，以两层混合求解（GA/SA 上界 + CP-SAT Hint 热启动）完成问题二至四。文件清单：`solver.py`、`heuristic.py`、`hybrid_solve.py`、`tune_params.py`、`extract.py`、`ga_q2.py`、`sa_q2.py`、`cross_validate.py`、`validate.py`、`make_figs.py`。

表 6 数据结构与基础参数变量说明

变量名	含义
TYPES	五类设备的键名列表（输送臂/清洗机/灌装机/传感机/抛光机）
NAME, PRICE	设备键名到中文名称、单价（元/台）的映射
G1	单班组各类设备台数；POOL 为双班组合并后台数
RAW	工序原始数据：(编号, 车间, {类型:(效率, 工程量)})
ORDER	各车间工序执行顺序（C 车间已含三遍循环）
DIST	班组/车间间距离（米），无向；SPEED 移动速度 2 m/s
trans(w1,w2)	由 w1 到 w2 的转运时间 $\lceil d/2 \rceil$ （同车间为 0）
build_ops	按设备台数与待修车间生成带工时的工序展开列表 ops 及车间工序链 jobs

表 7 CP-SAT 调度模型变量说明 (solve 函数)

变量名	含义
counts	各类设备总台数 (问题一/二取 G1, 问题三/四取池化并增购)
workshops	待整修车间集合; bases 初始转运取用的基地集合
S[i], E[i]	工序 i 的开工、完工时刻 (秒)
dur[t]	工序对类型 t 的占用时长 $p_{o,t}$
arc_*	资源路由布尔变量, 对应 $x_{o,o'}^r$; AddCircuit 构造单回路
init	某类设备首道工序的初始转运 (双基地取较大值)
Cmax	目标变量, 总完工时间; solver 为 CP-SAT 求解器对象
hints	可选热启动字典 (starts、cmax、routes), 经 AddHint 注入
solve_time	CP-SAT 求解耗时 (秒); used_hints 是否使用热启动

下列 solver.py 为 CP-SAT 核心; heuristic.py 为元启发式层; hybrid_solve.py 为混合求解入口; 问题四外层枚举在 hybrid_solve.py 主程序中实现。

Listing 1 solver.py: CP-SAT 核心 (含 hints 热启动)

```

1  # -*- coding: utf-8 -*-
2  """2026 校赛 B 题: 聚合柔性作业车间调度 (带跨车间转运) 的 CP-SAT 精确求解。
3  覆盖问题一~问题四的统一求解框架。"""
4  import math, sys, itertools, time
5  from ortools.sat.python import cp_model
6  sys.stdout.reconfigure(encoding="utf-8")
7
8  CEIL = math.ceil
9
10 # ----- 基础数据 -----
11 # 设备类型键
12 TYPES = ["arm", "wash", "fill", "sensor", "polish"]
13 NAME = {"arm": "自动化输送臂", "wash": "工业清洗机", "fill": "精密灌装机",
14         "sensor": "自动传感多功能机", "polish": "高速抛光机"}
15 PRICE = {"arm": 50000, "wash": 40000, "fill": 35000, "sensor": 80000, "polish":
16          75000}
17 # 单班组台数
18 G1 = {"arm": 4, "wash": 5, "fill": 5, "sensor": 1, "polish": 1}

```

```

19 # 工序: (编号, 车间, {类型: (效率 m3/h, 工程量 m3)})
20 RAW = [
21     ("A1", "A", {"fill": (200, 300), "arm": (250, 300)}),
22     ("A2", "A", {"polish": (100, 500), "wash": (250, 500)}),
23     ("A3", "A", {"sensor": (100, 500)}),
24     ("B1", "B", {"wash": (100, 120)}),
25     ("B2", "B", {"fill": (200, 1500), "arm": (300, 1500)}),
26     ("B3", "B", {"fill": (350, 360)}),
27     ("B4", "B", {"polish": (120, 360), "sensor": (100, 360)}),
28     ("C1", "C", {"wash": (250, 720), "arm": (250, 720)}),
29     ("C2", "C", {"fill": (350, 720)}),
30     ("C3", "C", {"fill": (200, 360), "arm": (250, 360)}),
31     ("C4", "C", {"polish": (120, 400), "wash": (100, 400)}),
32     ("C5", "C", {"sensor": (100, 400)}),
33     ("D1", "D", {"wash": (250, 600)}),
34     ("D2", "D", {"fill": (200, 800), "arm": (300, 800)}),
35     ("D3", "D", {"fill": (350, 450)}),
36     ("D4", "D", {"polish": (120, 1500), "sensor": (300, 1500)}),
37     ("D5", "D", {"sensor": (300, 1500)}),
38     ("D6", "D", {"polish": (100, 700)}),
39     ("E1", "E", {"wash": (250, 1000)}),
40     ("E2", "E", {"fill": (350, 600)}),
41     ("E3", "E", {"sensor": (300, 600), "wash": (100, 600)}),
42 ]
43
44 # 车间内工序顺序 (含 C 车间 C3-C5 重复 3 遍)
45 ORDER = {
46     "A": ["A1", "A2", "A3"],
47     "B": ["B1", "B2", "B3", "B4"],
48     "C": ["C1", "C2", "C3", "C4", "C5", "C3", "C4", "C5", "C3", "C4", "C5"],
49     "D": ["D1", "D2", "D3", "D4", "D5", "D6"],
50     "E": ["E1", "E2", "E3"],
51 }
52
53 # 距离表 (米), 无向
54 DIST = {}
55 def _d(a, b, v): DIST[(a, b)] = v; DIST[(b, a)] = v
56 for a, b, v in [("G1", "A", 400), ("G1", "B", 620), ("G1", "C", 460), ("G1", "D", 710), ("
57     "G1", "E", 400),
    ("G2", "A", 500), ("G2", "B", 460), ("G2", "C", 620), ("G2", "D", 680), ("

```

```

    G2", "E", 550),
58         ("A", "B", 1020), ("A", "C", 1050), ("A", "D", 900), ("A", "E", 1400),
59         ("B", "C", 1100), ("B", "D", 1630), ("B", "E", 720),
60         ("C", "D", 520), ("C", "E", 850), ("D", "E", 1030)]:
61     _d(a, b, v)
62 SPEED = 2 # m/s
63
64 def trans(w1, w2):
65     if w1 == w2: return 0
66     return CEIL(DIST[(w1, w2)] / SPEED)
67
68 # ----- 构造工序展开列表（带时长） -----
69 def build_ops(counts, workshops):
70     """counts: 每类设备总台数; workshops: 需整修车间列表。
71     返回 ops 列表, 每项 dict: id,name,ws,dur(每类秒数),precedes 链接由 jobs 给出。"""
72     raw = {r[0]: r for r in RAW}
73     ops = []
74     jobs = [] # 每个车间一条工序链(ops 下标序列)
75     for w in workshops:
76         chain = []
77         for code in ORDER[w]:
78             _, ws, reqs = raw[code]
79             dur = {}
80             for t, (eff, vol) in reqs.items():
81                 dur[t] = CEIL(vol / (eff * counts[t])) * 3600
82             idx = len(ops)
83             ops.append({"id": code, "ws": ws, "dur": dur, "k": len(ops)})
84             chain.append(idx)
85         jobs.append(chain)
86     return ops, jobs
87
88 # ----- CP-SAT 求解 -----
89 def solve(counts, workshops, bases, time_limit=60, log=False, hints=None):
90     """bases: 车间初始转运取的基地集合(单班组 ['G1']; 双班组 ['G1', 'G2'] 取 max)
91     。
92     hints: 可选热启动字典 {"starts": [...], "cmax": int, "routes": {t: {(i,j):
93     0/1}}},
94     仅作搜索引导, 不裁剪最优解。"""
95     ops, jobs = build_ops(counts, workshops)

```

```

94     n = len(ops)
95     depot = n
96     H = sum(max(o["dur"].values()) for o in ops) + 50000 # 时间上界
97     m = cp_model.CpModel()
98     S = [m.NewIntVar(0, H, f"S{i}") for i in range(n)]
99     E = [m.NewIntVar(0, H, f"E{i}") for i in range(n)]
100     for i, o in enumerate(ops):
101         md = max(o["dur"].values())
102         m.Add(E[i] == S[i] + md)
103     # 车间内工序先后
104     for chain in jobs:
105         for a, b in zip(chain, chain[1:]):
106             m.Add(S[b] >= E[a])
107     # 每类设备(聚合为一台超级机器)的路由: circuit + 转运
108     arc_lit = {} # (t, i, j) -> BoolVar
109     for t in TYPES:
110         users = [i for i, o in enumerate(ops) if t in o["dur"]]
111         if not users: continue
112         arcs = []
113         for i in users:
114             b0 = m.NewBoolVar(f"arc_{t}_dep_{i}")
115             arcs.append((depot, i, b0)); arc_lit[(t, depot, i)] = b0
116             init = max(trans(base, ops[i]["ws"]) for base in bases)
117             m.Add(S[i] >= init).OnlyEnforceIf(b0)
118             b1 = m.NewBoolVar(f"arc_{t}_{i}_dep")
119             arcs.append((i, depot, b1)); arc_lit[(t, i, depot)] = b1
120             for j in users:
121                 if i == j: continue
122                 b = m.NewBoolVar(f"arc_{t}_{i}_{j}")
123                 arcs.append((i, j, b)); arc_lit[(t, i, j)] = b
124                 m.Add(S[j] >= S[i] + ops[i]["dur"][t] + trans(ops[i]["ws"], ops
125                     [j]["ws"])).OnlyEnforceIf(b)
126             m.AddCircuit(arcs)
127         Cmax = m.NewIntVar(0, H, "Cmax")
128         for i in range(n):
129             m.Add(Cmax >= E[i])
130     m.Minimize(Cmax)
131     used_hints = False
132     if hints:
133         used_hints = True

```

```

133     for i, s in enumerate(hints["starts"]):
134         m.AddHint(S[i], int(s))
135         m.AddHint(E[i], int(s) + max(ops[i]["dur"].values()))
136     m.AddHint(Cmax, int(hints["cmax"]))
137     for t, arcs in hints.get("routes", {}).items():
138         for (i, j), val in arcs.items():
139             key = (t, i, j)
140             if key in arc_lit:
141                 m.AddHint(arc_lit[key], int(val))
142     solver = cp_model.CpSolver()
143     solver.parameters.max_time_in_seconds = time_limit
144     solver.parameters.num_search_workers = 8
145     if log: solver.parameters.log_search_progress = True
146     t0 = time.perf_counter()
147     st = solver.Solve(m)
148     solve_time = time.perf_counter() - t0
149     res = {"status": solver.StatusName(st), "cmax": solver.Value(Cmax) if st in
150           (cp_model.OPTIMAL, cp_model.FEASIBLE) else None,
151           "obj_bound": solver.BestObjectiveBound, "ops": ops, "solve_time":
152           solve_time, "used_hints": used_hints}
153     if res["cmax"] is not None:
154         sched = []
155         for i, o in enumerate(ops):
156             sched.append({"id": o["id"], "ws": o["ws"], "start": solver.Value(S
157                             [i]),
158                             "end": solver.Value(E[i]), "dur": {t: o["dur"][t] for
159                             t in o["dur"]}})
160         res["sched"] = sched
161     return res
162
163 def show(tag, res):
164     print(f"\n===== {tag} : status={res['status']} Cmax={res['cmax']} =====")
165
166 POOL = {t: 2 * G1[t] for t in TYPES} # 双班组合并台数
167
168 if __name__ == "__main__":
169     ALL = ["A", "B", "C", "D", "E"]
170     r1 = solve(G1, ["A"], ["G1"], time_limit=20)
171     show("Q1 (A only)", r1)
172     from hybrid_solve import solve_hybrid, print_hybrid

```

```

169     r2 = solve_hybrid(G1, ALL, ["G1"], time_limit=120)
170     print_hybrid("Q2", r2)
171     r3 = solve_hybrid(P00L, ALL, ["G1", "G2"], time_limit=180)
172     print_hybrid("Q3", r3)
173     best = None
174     for kp in range(0, 5):
175         for ks in range(0, 5):
176             cost = kp * PRICE["polish"] + ks * PRICE["sensor"]
177             if cost > 500000: continue
178             cc = dict(P00L); cc["polish"] += kp; cc["sensor"] += ks
179             rh = solve_hybrid(cc, ALL, ["G1", "G2"], time_limit=40,
cold_time_limit=40)
180             r = rh["hot"]
181             if r["cmax"] is None: continue
182             print(f"    buy polish+{kp} sensor+{ks} cost={cost} Cmax={r['cmax']}
({r['status']})")
183             key = (r["cmax"], cost)
184             if best is None or key < best[0]:
185                 best = (key, kp, ks, rh)
186             (cm, cost), kp, ks, r4 = best
187             print(f"\n>>> Q4 best: polish+{kp}, sensor+{ks}, cost={cost}, Cmax={cm}")

```

Listing 2 heuristic.py: GA/SA 解码与路由提取

```

1  # -*- coding: utf-8 -*-
2  """元启发式层：遗传算法 + 模拟退火，为 CP-SAT 提供可行上界与热启动 Hint。
3  解码口径与正文聚合模型一致：双设备取 max 完工、编组并行转场、设备提前释放。"""
4  import random, math, time, sys
5  from solver import build_ops, trans, TYPES
6  sys.stdout.reconfigure(encoding="utf-8")
7
8  # 推荐参数（由 tune_params.py 在 Q2 上网格搜索后确定，此处为默认）
9  DEFAULT_GA = {"pop_size": 150, "gens": 800, "pm": 0.2, "seed": 0}
10 DEFAULT_SA = {"T0": 4000.0, "Tend": 1e-2, "alpha": 0.97, "iters": 400, "seed":
    0}
11
12
13 def make_template(jobs):
14     tpl = []
15     for w in range(len(jobs)):

```

```

16         tpl += [w] * len(jobs[w])
17     return tpl
18
19
20 def decode_full(chrom, ops, jobs, bases):
21     """串行解码, 返回 (makespan, 各工序开工时刻列表)。"""
22     free = {t: 0 for t in TYPES}
23     loc = {t: None for t in TYPES}
24     ptr = [0] * len(jobs)
25     done = [0] * len(jobs)
26     starts = [0] * len(ops)
27     cmax = 0
28     for w in chrom:
29         o = jobs[w][ptr[w]]; ptr[w] += 1
30         ws = ops[o]["ws"]; dur = ops[o]["dur"]
31         start = done[w]
32         for t in dur:
33             if loc[t] is None:
34                 ready = free[t] + max(trans(b, ws) for b in bases)
35             else:
36                 ready = free[t] + trans(loc[t], ws)
37             start = max(start, ready)
38         for t in dur:
39             free[t] = start + dur[t]; loc[t] = ws
40         comp = start + max(dur.values())
41         done[w] = comp; starts[o] = start; cmax = max(cmax, comp)
42     return cmax, starts
43
44
45 def extract_routes(starts, ops, n_depot):
46     """由开工时刻反推各资源访问序, 生成 AddCircuit 弧 Hint (1=选中)。"""
47     routes = {}
48     for t in TYPES:
49         users = sorted([i for i, o in enumerate(ops) if t in o["dur"]],
50                        key=lambda i: starts[i])
51         if not users:
52             continue
53         arcs = {}
54         arcs[(n_depot, users[0])] = 1
55         for a, b in zip(users, users[1:]):

```

```

56         arcs[(a, b)] = 1
57         arcs[(users[-1], n_depot)] = 1
58         routes[t] = arcs
59     return routes
60
61
62 def pox(p1, p2, nw):
63     jset = set(random.sample(range(nw), random.randint(1, nw - 1)))
64     child = [g if g in jset else None for g in p1]
65     fill = [g for g in p2 if g not in jset]
66     k = 0
67     for i in range(len(child)):
68         if child[i] is None:
69             child[i] = fill[k]; k += 1
70     return child
71
72
73 def ga(ops, jobs, bases, pop_size=150, gens=800, pm=0.2, seed=0):
74     random.seed(seed)
75     nw = len(jobs); tpl = make_template(jobs)
76     pop = [random.sample(tpl, len(tpl)) for _ in range(pop_size)]
77     fit = [decode_full(c, ops, jobs, bases)[0] for c in pop]
78     bi = min(range(pop_size), key=lambda i: fit[i])
79     bc, bf = pop[bi][:], fit[bi]
80     for _ in range(gens):
81         npop, nfit = [bc[:]], [bf]
82         while len(npop) < pop_size:
83             a = pop[min(random.sample(range(pop_size), 3), key=lambda i: fit[i
84 ]))]
85             b = pop[min(random.sample(range(pop_size), 3), key=lambda i: fit[i
86 ]))]
87             c = pox(a, b, nw)
88             if random.random() < pm:
89                 i, j = random.sample(range(len(c)), 2); c[i], c[j] = c[j], c[i]
90                 npop.append(c); nfit.append(decode_full(c, ops, jobs, bases)[0])
91             pop, fit = npop, nfit
92             i = min(range(pop_size), key=lambda j: fit[j])
93             if fit[i] < bf:
94                 bf, bc = fit[i], pop[i][:]
95     return bf, bc, decode_full(bc, ops, jobs, bases)[1]

```

```

94
95
96 def sa(ops, jobs, bases, chrom=None, T0=4000.0, Tend=1e-2, alpha=0.97, iters
    =400, seed=0):
97     random.seed(seed)
98     tpl = make_template(jobs)
99     cur = chrom[:] if chrom else random.sample(tpl, len(tpl))
100     cur_f = decode_full(cur, ops, jobs, bases)[0]
101     best, best_f = cur[:], cur_f
102     T = T0
103     while T > Tend:
104         for _ in range(iters):
105             nb = cur[:]
106             if random.random() < 0.5:
107                 i, j = random.sample(range(len(nb)), 2); nb[i], nb[j] = nb[j],
nb[i]
108             else:
109                 i = random.randrange(len(nb)); g = nb.pop(i)
110                 nb.insert(random.randrange(len(nb) + 1), g)
111                 f = decode_full(nb, ops, jobs, bases)[0]
112                 if f <= cur_f or random.random() < math.exp((cur_f - f) / T):
113                     cur, cur_f = nb, f
114                     if f < best_f:
115                         best, best_f = nb[:], f
116             T *= alpha
117     return best_f, best, decode_full(best, ops, jobs, bases)[1]
118
119
120 def run_metaheuristic(ops, jobs, bases, ga_params=None, sa_params=None):
121     """GA 求初解 → SA 局部改良, 返回 (上界, starts, chrom, 耗时秒)。"""
122     ga_params = ga_params or DEFAULT_GA
123     sa_params = sa_params or DEFAULT_SA
124     t0 = time.perf_counter()
125     gf, gchrom, gstarts = ga(ops, jobs, bases, **ga_params)
126     sf, schrom, sstarts = sa(ops, jobs, bases, chrom=gchrom, **sa_params)
127     elapsed = time.perf_counter() - t0
128     if sf <= gf:
129         return sf, sstarts, schrom, elapsed
130     return gf, gstarts, gchrom, elapsed

```

Listing 3 hybrid_solve.py: 两层混合求解入口

```

1  # -*- coding: utf-8 -*-
2  """两层混合求解: GA/SA 求可行上界 → CP-SAT Hint 热启动精确证明。"""
3  import sys, time
4  from solver import solve, build_ops, G1, POOL, PRICE
5  from heuristic import run_metaheuristic, extract_routes, DEFAULT_GA, DEFAULT_SA
6  from validate import validate
7  sys.stdout.reconfigure(encoding="utf-8")
8
9
10 def solve_hybrid(counts, workshops, bases, ga_params=None, sa_params=None,
11                  time_limit=120, cold_time_limit=None, log=False):
12     """返回混合求解全过程指标。"""
13     ga_params = ga_params or DEFAULT_GA
14     sa_params = sa_params or DEFAULT_SA
15     cold_time_limit = cold_time_limit or time_limit
16     ops, jobs = build_ops(counts, workshops)
17     n = len(ops)
18     depot = n
19
20     ub, starts, chrom, meta_time = run_metaheuristic(ops, jobs, bases,
21                                                       ga_params, sa_params)
22     ok, msg, _ = validate(starts, ops, jobs, bases)
23     routes = extract_routes(starts, ops, depot)
24     hints = {"starts": starts, "cmax": ub, "routes": routes}
25
26     cold = solve(counts, workshops, bases, time_limit=cold_time_limit, log=log)
27     hot = solve(counts, workshops, bases, time_limit=time_limit, hints=hints,
28                log=log)
29
30     return {
31         "ub": ub, "chrom": chrom, "meta_time": meta_time,
32         "meta_feasible": ok, "meta_msg": msg,
33         "cold": cold, "hot": hot,
34         "optimal": hot["cmax"],
35         "gap_ub": (ub - hot["cmax"]) if hot["cmax"] is not None else None,
36     }

```

```

37 def print_hybrid(tag, r):
38     print(f"\n===== {tag} 混合求解 =====")
39     print(f" 元启发式上界: {r['ub']} s (可行={r['meta_feasible']}, 耗时={r['meta_time']:.1f}s)")
40     print(f" CP-SAT 冷启动: Cmax={r['cold']['cmax']} status={r['cold']['status']} ")
41     print(f" 耗时={r['cold']['solve_time']:.1f}s")
42     print(f" CP-SAT 热启动: Cmax={r['hot']['cmax']} status={r['hot']['status']} ")
43     print(f" 耗时={r['hot']['solve_time']:.1f}s")
44     if r['cold']['solve_time'] > 0:
45         ratio = r['cold']['solve_time'] / max(r['hot']['solve_time'], 0.01)
46         print(f" 加速比(冷/热): {ratio:.2f}x")
47     if r['gap_ub'] is not None:
48         print(f" 上界间隙: {r['gap_ub']} s")
49
50
51 if __name__ == "__main__":
52     ALL = ["A", "B", "C", "D", "E"]
53     r2 = solve_hybrid(G1, ALL, ["G1"], time_limit=120)
54     print_hybrid("Q2", r2)
55     r3 = solve_hybrid(P00L, ALL, ["G1", "G2"], time_limit=180)
56     print_hybrid("Q3", r3)
57     cc = dict(P00L); cc["polish"] += 3; cc["sensor"] += 3
58     r4 = solve_hybrid(cc, ALL, ["G1", "G2"], time_limit=60)
59     print_hybrid("Q4", r4)
60     # Q4 全枚举 (购置组合)
61     print("\n===== Q4 购置枚举 (混合求解) =====")
62     best = None
63     for kp in range(0, 5):
64         for ks in range(0, 5):
65             cost = kp * PRICE["polish"] + ks * PRICE["sensor"]
66             if cost > 500000: continue
67             cc2 = dict(P00L); cc2["polish"] += kp; cc2["sensor"] += ks
68             rh = solve_hybrid(cc2, ALL, ["G1", "G2"], time_limit=40,
cold_time_limit=40)
69             cm = rh["optimal"]
70             if cm is None: continue
71             print(f" polish+{kp} sensor+{ks} cost={cost} Cmax={cm} ({rh['hot']
['status']}")

```

```

72         key = (cm, cost)
73         if best is None or key < best[0]:
74             best = (key, kp, ks, rh)
75     if best:
76         (cm, cost), kp, ks, rh = best
77     print(f"\n>>> Q4 best: polish+{kp}, sensor+{ks}, cost={cost}, Cmax={cm}
")

```

Listing 4 tune_params.py: 网格调参与冷/热对比

```

1  # -*- coding: utf-8 -*-
2  """Q2 上网格搜索 GA/SA 参数, 输出调参表与冷/热启动对比数据。"""
3  import sys, itertools, json
4  from solver import solve, G1
5  from heuristic import run_metaheuristic, extract_routes
6  sys.stdout.reconfigure(encoding="utf-8")
7
8  ALL = ["A", "B", "C", "D", "E"]
9  BASES = ["G1"]
10 OPTIMAL = 144880
11 COLD_LIMIT = 120
12 HOT_LIMIT = 120
13
14 GA_GRID = {
15     "pop_size": [80, 150, 200],
16     "gens": [400, 800],
17     "pm": [0.15, 0.2],
18 }
19 SA_GRID = {
20     "TO": [3000.0, 4000.0],
21     "alpha": [0.95, 0.97],
22     "iters": [300, 400],
23 }
24 SA_FIXED = {"Tend": 1e-2, "seed": 0}
25 GA_FIXED = {"seed": 0}
26
27
28 def tune():
29     from solver import build_ops
30     ops, jobs = build_ops(G1, ALL)

```

```

31 depot = len(ops)
32 rows = []
33 ga_keys = list(GA_GRID.keys())
34 sa_keys = list(SA_GRID.keys())
35 for ga_vals in itertools.product(*GA_GRID.values()):
36     ga_p = dict(zip(ga_keys, ga_vals)); ga_p.update(GA_FIXED)
37     for sa_vals in itertools.product(*SA_GRID.values()):
38         sa_p = dict(zip(sa_keys, sa_vals)); sa_p.update(SA_FIXED)
39         ub, starts, _, meta_t = run_metaheuristic(ops, jobs, BASES, ga_p,
sa_p)
40         routes = extract_routes(starts, ops, depot)
41         hints = {"starts": starts, "cmax": ub, "routes": routes}
42         cold = solve(G1, ALL, BASES, time_limit=COLD_LIMIT)
43         hot = solve(G1, ALL, BASES, time_limit=HOT_LIMIT, hints=hints)
44         gap = ub - OPTIMAL
45         speedup = cold["solve_time"] / max(hot["solve_time"], 0.01)
46         rows.append({
47             "ga": ga_p, "sa": sa_p,
48             "ub": ub, "gap": gap, "meta_time": meta_t,
49             "cold_time": cold["solve_time"], "hot_time": hot["solve_time"],
50             "speedup": speedup,
51         })
52         print(f"GA pop={ga_p['pop_size']} gens={ga_p['gens']} pm={ga_p['pm
53             f"SA T0={sa_p['T0']} alpha={sa_p['alpha']} iters={sa_p['iters
54             f"ub={ub} gap={gap} meta={meta_t:.1f}s cold={cold['solve_time
55             f"hot={hot['solve_time']:.1f}s speedup={speedup:.2f}x", flush
= True)
56 # 推荐: makespan 优先, 耗时次之
57 rows.sort(key=lambda r: (r["gap"], r["meta_time"] + r["hot_time"]))
58 best = rows[0]
59 print("\n===== 推荐参数 =====")
60 print("GA:", best["ga"])
61 print("SA:", best["sa"])
62 print(f"上界={best['ub']} 间隙={best['gap']} 冷={best['cold_time']:.1f}s 热
={best['hot_time']:.1f}s "
63         f"加速={best['speedup']:.2f}x")
64 with open("tune_results.json", "w", encoding="utf-8") as f:

```

```

65     json.dump({"rows": rows, "recommended": best}, f, ensure_ascii=False,
66     indent=2)
67
68     return rows, best
69
70 def cold_hot_table():
71     """Q2/Q3/Q4 冷/热启动对比（用推荐参数）。"""
72     from hybrid_solve import solve_hybrid
73     from solver import POOL, PRICE
74     results = []
75     r2 = solve_hybrid(G1, ALL, BASES, time_limit=120)
76     results.append(("Q2", r2))
77     r3 = solve_hybrid(POOL, ALL, ["G1", "G2"], time_limit=180)
78     results.append(("Q3", r3))
79     cc = dict(POOL); cc["polish"] += 3; cc["sensor"] += 3
80     r4 = solve_hybrid(cc, ALL, ["G1", "G2"], time_limit=60)
81     results.append(("Q4", r4))
82     print("\n===== 冷/热启动对比表 =====")
83     for tag, r in results:
84         cold_t = r["cold"]["solve_time"]
85         hot_t = r["hot"]["solve_time"]
86         sp = cold_t / max(hot_t, 0.01)
87         print(f"{tag}: 冷={cold_t:.1f}s 热={hot_t:.1f}s 加速={sp:.2f}x 最优={r['optimal']}")
88     with open("cold_hot.json", "w", encoding="utf-8") as f:
89         json.dump({tag: {"cold": r["cold"]["solve_time"], "hot": r["hot"]["solve_time"],
90             "optimal": r["optimal"], "ub": r["ub"]} for tag, r in results},
91             f, ensure_ascii=False, indent=2)
92     return results
93
94 if __name__ == "__main__":
95     import argparse
96     p = argparse.ArgumentParser()
97     p.add_argument("--quick", action="store_true", help="仅跑推荐参数冷/热对比")
98     args = p.parse_args()
99     if args.quick:
100         cold_hot_table()

```

```

101     else:
102         tune()
103         cold_hot_table()

```

下列 extract.py 用于抽取各问最优解的逐设备调度时间线（生成正文表 9-12 的明细）。

Listing 5 extract.py: 调度明细抽取程序

```

1  # -*- coding: utf-8 -*-
2  """抽取各问最优解的逐设备调度明细，按设备类型给出有序时间线，供结果表使用。"""
3  import sys
4  from solver import solve, G1, POOL, TYPES, NAME, trans
5  sys.stdout.reconfigure(encoding="utf-8")
6
7  def hhmmss(s):
8      return f"{s//3600:02d}:{(s%3600)//60:02d}:{s%60:02d}"
9
10 def timeline(res, label, bases):
11     print(f"\n##### {label} Cmax={res['cmax']} #####")
12     sched = {s["id"]+"_"+str(k): s for k, s in enumerate(res["sched"])}
13     items = res["sched"]
14     for t in TYPES:
15         users = [s for s in items if t in s["dur"]]
16         if not users: continue
17         users.sort(key=lambda s: s["start"])
18         print(f"\n--- {NAME[t]} ({t}) ---")
19         prev_ws = None
20         for s in users:
21             d = s["dur"][t]
22             st, en = s["start"], s["start"] + d
23             mv = ""
24             if prev_ws is None:
25                 init = max(trans(b, s["ws"]) for b in bases)
26                 mv = f"[init->{s['ws']}] 转运{init}s"
27             elif prev_ws != s["ws"]:
28                 mv = f"[{prev_ws}->{s['ws']}] 转运{trans(prev_ws,s['ws'])}s"
29             print(f"  {s['ws']}-{s['id']:>3}  {hhmmss(st)} -> {hhmmss(en)}  dur
30                   ={d:>6}s {mv}")
31             prev_ws = s["ws"]
32
33 ALL = ["A", "B", "C", "D", "E"]

```

```

33 r1 = solve(G1, ["A"], ["G1"], 20); timeline(r1, "Q1", ["G1"])
34 r2 = solve(G1, ALL, ["G1"], 120); timeline(r2, "Q2", ["G1"])
35 r3 = solve(P00L, ALL, ["G1", "G2"], 180); timeline(r3, "Q3", ["G1", "G2"])
36 cc = dict(P00L); cc["polish"] += 3; cc["sensor"] += 3
37 r4 = solve(cc, ALL, ["G1", "G2"], 60); timeline(r4, "Q4 (polish+3,sensor+3)", [
    "G1", "G2"])

```

混合求解对照与可行性校验

ga_q2.py、sa_q2.py 调用 heuristic.py.validate.py 为独立可行性校验器。cross_validate.py 输出混合求解对照表。各文件关键变量见表 8。

表 8 元启发式与混合求解程序变量说明

变量/函数	含义
decode_full	串行调度解码, 返回 makespan 与各工序开工时刻
extract_routes	由开工时刻反推各资源访问序, 生成路由弧 Hint
run_metaheuristic	GA → SA 流程, 返回上界、starts、耗时
solve_hybrid	混合求解入口: 元启发式 + 冷/热 CP-SAT
pox	优先保留顺序交叉, 保持多重集不变与先后可行
neighbor	模拟退火邻域: 两点交换或单点重插入
T0,Tend,alpha,iter	模拟退火初温、终温、降温系数、每温迭代次数
validate	可行性校验, 返回 (是否可行, 违规明细, makespan)

Listing 6 ga_q2.py: 遗传算法 (调用 heuristic)

```

1  # -*- coding: utf-8 -*-
2  """问题二: 遗传算法(GA)——调用统一元启发式模块。"""
3  import sys
4  from solver import build_ops, G1
5  from heuristic import ga, DEFAULT_GA
6  sys.stdout.reconfigure(encoding="utf-8")
7
8  WORKSHOPS = ["A", "B", "C", "D", "E"]
9  OPS, JOBS = build_ops(G1, WORKSHOPS)
10
11 if __name__ == "__main__":

```

```

12     p = DEFAULT_GA.copy()
13     best_f, best_c, starts = ga(OPS, JOBS, ["G1"], **p)
14     print("GA 最优 makespan =", best_f, "s")
15     print("(CP-SAT 精确最优 = 144880 s)")

```

Listing 7 sa_q2.py: 模拟退火 (调用 heuristic)

```

1  # -*- coding: utf-8 -*-
2  """问题二：模拟退火(SA)——调用统一元启发式模块。"""
3  import sys
4  from solver import build_ops, G1
5  from heuristic import sa, DEFAULT_SA
6  sys.stdout.reconfigure(encoding="utf-8")
7
8  WORKSHOPS = ["A", "B", "C", "D", "E"]
9  OPS, JOBS = build_ops(G1, WORKSHOPS)
10
11 if __name__ == "__main__":
12     p = DEFAULT_SA.copy()
13     best_f, best_c, starts = sa(OPS, JOBS, ["G1"], **p)
14     print("SA 最优 makespan =", best_f, "s")
15     print("(CP-SAT 精确最优 = 144880 s)")

```

Listing 8 validate.py: 可行性校验器

```

1  # -*- coding: utf-8 -*-
2  """调度方案可行性校验器（独立于求解器，用于杜绝"看起来短但不可行"的解）。
3  检查四类约束：
4      1) 车间内工序先后：后序开工 >= 前序完工（完工取双设备较晚者）；
5      2) 同类设备不重叠：同一类型设备的各占用区间两两不相交；
6      3) 转运充足：同类相邻作业跨车间时，间隔 >= 转运时间；首作业 >= 由基地出发的转
       运；
7      4) 双设备完工：工序完工 = 两类设备结束时间的较大值。
8  适用于 CP-SAT、遗传算法、模拟退火三种来源的解。"""
9  import sys
10 from solver import build_ops, trans, TYPES, NAME
11 sys.stdout.reconfigure(encoding="utf-8")
12
13
14 def validate(starts, ops, jobs, bases):
15     """starts: 按工序下标给出的开工时刻列表。

```

```

16     返回 (是否可行, 违规明细列表, makespan)。"""
17     n = len(ops)
18     end = [starts[i] + max(ops[i]["dur"].values()) for i in range(n)] # 双设备
    取较晚者
19     vio = []
20
21     # (1) 车间内工序先后
22     for chain in jobs:
23         for a, b in zip(chain, chain[1:]):
24             if starts[b] < end[a]:
25                 vio.append(f"先后违规: {ops[a]['id']} (完工{end[a]}) -> {ops[b]
    ['id']} (开工{starts[b]})")
26
27     # (2)(3) 各类设备: 不重叠 + 转运充足
28     for t in TYPES:
29         users = [i for i in range(n) if t in ops[i]["dur"]]
30         users.sort(key=lambda i: starts[i])
31         if users:
32             init = max(trans(base, ops[users[0]]["ws"]) for base in bases)
33             if starts[users[0]] < init:
34                 vio.append(f"初始转运不足 [{NAME[t]}]: 首作业{ops[users[0]]['id']
    ['']} 开工{starts[users[0]]} < 需{init}")
35             for x, y in zip(users, users[1:]):
36                 x_end = starts[x] + ops[x]["dur"][t] # 该设备完成本职工
    的时刻
37                 if starts[y] < x_end:
38                     vio.append(f"设备重叠 [{NAME[t]}]: {ops[x]['id']} 与 {ops[y]['id']
    ['']} 时间重叠")
39                 else:
40                     need = x_end + trans(ops[x]["ws"], ops[y]["ws"])
41                     if starts[y] < need:
42                         vio.append(f"转运不足 [{NAME[t]}]: {ops[x]['id']} ({ops[x]['
    ws']}) -> "
43                                     f"{ops[y]['id']} ({ops[y]['ws']}) 间隔不足, 需到{
    need} 实到{starts[y]}")
44
45     return (len(vio) == 0, vio, max(end))
46
47
48 def validate_result(res, counts, workshops, bases):

```

```

49     """校验 solver.solve 返回的 CP-SAT 解。"""
50     ops, jobs = build_ops(counts, workshops)
51     starts = [s["start"] for s in res["sched"]] # sched 与 build_ops 同序
52     return validate(starts, ops, jobs, bases)
53
54
55 if __name__ == "__main__":
56     from solver import solve, G1, POOL, PRICE
57     ALL = ["A", "B", "C", "D", "E"]
58     cases = [
59         ("Q2", G1, ALL, ["G1"]),
60         ("Q3", POOL, ALL, ["G1", "G2"]),
61     ]
62     cc = dict(POOL); cc["polish"] += 3; cc["sensor"] += 3
63     cases.append(("Q4", cc, ALL, ["G1", "G2"]))
64     for tag, counts, ws, bases in cases:
65         res = solve(counts, ws, bases, time_limit=120)
66         ok, vio, cmax = validate_result(res, counts, ws, bases)
67         print(f"{tag}: CP-SAT Cmax={res['cmax']} 校验makespan={cmax} 可行={ok}
68             违规数={len(vio)}")
69         for v in vio[:5]:
70             print("    -", v)

```

Listing 9 cross_validate.py: 混合求解结果对照

```

1  # -*- coding: utf-8 -*-
2  """混合求解对照: 纯 CP-SAT / 混合(GA+SA+Hint) / 仅 GA 上界。
3  对问题二/三/四分别输出对照表, 并用 validate.py 校验可行性。"""
4  import sys
5  from solver import solve, build_ops, G1, POOL, PRICE
6  from heuristic import run_metaheuristic, DEFAULT_GA, DEFAULT_SA
7  from validate import validate
8  sys.stdout.reconfigure(encoding="utf-8")
9
10
11 def compare_case(tag, counts, workshops, bases, exact):
12     ops, jobs = build_ops(counts, workshops)
13     ub, starts, _, meta_t = run_metaheuristic(ops, jobs, bases)
14     gok, _, _ = validate(starts, ops, jobs, bases)
15     cold = solve(counts, workshops, bases, time_limit=120)

```

```

16     from heuristic import extract_routes
17     depot = len(ops)
18     routes = extract_routes(starts, ops, depot)
19     hints = {"starts": starts, "cmax": ub, "routes": routes}
20     hot = solve(counts, workshops, bases, time_limit=120, hints=hints)
21     print(f"{tag}: 上界(GA+SA)={ub}(可行={gok}, {meta_t:.1f}s) | "
22           f"冷启动={cold['cmax']}({cold['solve_time']:.1f}s) | "
23           f"热启动={hot['cmax']}({hot['solve_time']:.1f}s, {hot['status']}) | "
24           f"间隙={ub - exact}")
25     return {"tag": tag, "ub": ub, "exact": exact, "gap": ub - exact,
26            "cold_time": cold["solve_time"], "hot_time": hot["solve_time"],
27            "feasible": gok, "status": hot["status"]}
28
29
30 if __name__ == "__main__":
31     ALL = ["A", "B", "C", "D", "E"]
32     cc = dict(POOL); cc["polish"] += 3; cc["sensor"] += 3
33     print("==== 表 C 混合求解结果对照 =====")
34     compare_case("Q2", G1, ALL, ["G1"], 144880)
35     compare_case("Q3", POOL, ALL, ["G1", "G2"], 73575)
36     compare_case("Q4", cc, ALL, ["G1", "G2"], 32608)

```

图表生成程序

make_figs.py 调用 solver.solve 取最优调度，生成正文甘特图与热力图。

Listing 10 make_figs.py: 甘特图与热力图生成程序

```

1  # -*- coding: utf-8 -*-
2  """生成论文图表: Q2/Q3/Q4 甘特图、资源载荷热力图、设备利用率热力图、问题四购置收益
   热力图。
3  图片输出至 figs/ 目录。"""
4  import os, sys
5  import numpy as np
6  import matplotlib
7  matplotlib.use("Agg")
8  import matplotlib.pyplot as plt
9  from matplotlib.patches import Patch
10 from solver import build_ops, solve, G1, POOL, PRICE, TYPES, NAME
11 sys.stdout.reconfigure(encoding="utf-8")
12

```

```

13 # ----- 中文字体（不可用则回退英文标签） -----
14 ZH = True
15 for f in ["Microsoft YaHei", "SimHei", "SimSun", "KaiTi"]:
16     try:
17         matplotlib.font_manager.findfont(f, fallback_to_default=False)
18         plt.rcParams["font.sans-serif"] = [f]; plt.rcParams["axes.unicode_minus"] = False
19         break
20     except Exception:
21         continue
22 else:
23     ZH = False
24
25 EN = {"arm": "Arm", "wash": "Washer", "fill": "Filler", "sensor": "Sensor", "polish": "Polisher"}
26 def lab(t): return NAME[t] if ZH else EN[t]
27
28 ALL = ["A", "B", "C", "D", "E"]
29 WS_COLOR = {"A": "#4E79A7", "B": "#59A14F", "C": "#E15759", "D": "#F28E2B", "E": "#9C6ADE"}
30 os.makedirs("figs", exist_ok=True)
31
32
33 def gantt(res, title, path):
34     sched = res["sched"]
35     fig, ax = plt.subplots(figsize=(11, 4.2))
36     for yi, t in enumerate(TYPES):
37         for s in sched:
38             if t in s["dur"]:
39                 x0 = s["start"] / 3600.0; w = s["dur"][t] / 3600.0
40                 ax.broken_barh([(x0, w)], (yi * 10 + 1, 8),
41                               facecolors=WS_COLOR[s["ws"]], edgecolor="white",
42                               linewidth=0.5)
43                 if w * 3600 >= 2500:
44                     ax.text(x0 + w / 2, yi * 10 + 5, s["id"], ha="center", va="center",
45                             fontsize=6.5, color="white")
46     ax.set_yticks([yi * 10 + 5 for yi in range(len(TYPES))])
47     ax.set_yticklabels([lab(t) for t in TYPES])
48     ax.set_xlabel("时间 / h" if ZH else "Time / h")

```

```

48     ax.set_title(title)
49     ax.set_xlim(0, max(s["end"] for s in sched) / 3600.0 * 1.02)
50     ax.grid(axis="x", linestyle=":", alpha=0.4)
51     ax.legend(handles=[Patch(facecolor=WS_COLOR[w], label=("%s车间" % w) if ZH
else ("WS %s" % w)) for w in ALL],
52                 ncol=5, loc="upper center", bbox_to_anchor=(0.5, -0.16), fontsize
=8, frameon=False)
53     fig.tight_layout(); fig.savefig(path, dpi=150, bbox_inches="tight"); plt.
close(fig)
54     print("saved", path)
55
56
57 def load_heatmap(path):
58     ops, _ = build_ops(G1, ALL)
59     M = np.zeros((len(TYPES), len(ALL)))
60     for o in ops:
61         wj = ALL.index(o["ws"])
62         for t in o["dur"]:
63             M[TYPES.index(t), wj] += o["dur"][t]
64     M /= 3600.0
65     fig, ax = plt.subplots(figsize=(7, 4))
66     im = ax.imshow(M, cmap="YlOrRd", aspect="auto")
67     ax.set_xticks(range(len(ALL))); ax.set_xticklabels([("%s车间" % w) if ZH
else w for w in ALL])
68     ax.set_yticks(range(len(TYPES))); ax.set_yticklabels([lab(t) for t in TYPES
])
69     for i in range(len(TYPES)):
70         for j in range(len(ALL)):
71             if M[i, j] > 0:
72                 ax.text(j, i, f"{M[i, j]:.1f}", ha="center", va="center",
fontsize=8,
73                        color="black" if M[i, j] < M.max() * 0.6 else "white")
74     ax.set_title("各类设备在各车间的作业载荷 / h (单班组)" if ZH else "Resource
load by workshop / h")
75     fig.colorbar(im, ax=ax, label="作业载荷 / h" if ZH else "load / h")
76     fig.tight_layout(); fig.savefig(path, dpi=150); plt.close(fig)
77     print("saved", path)
78
79
80 def util_heatmap(path):

```

```

81 cc = dict(POOL); cc["polish"] += 3; cc["sensor"] += 3
82 cases = [("Q2", G1, ["G1"]), ("Q3", POOL, ["G1", "G2"]), ("Q4", cc, ["G1",
83 "G2"])]
84 M = np.zeros((len(TYPES), len(cases)))
85 for cj, (tag, counts, bases) in enumerate(cases):
86     ops, _ = build_ops(counts, ALL)
87     res = solve(counts, ALL, bases, time_limit=120)
88     cmax = res["cmax"]
89     busy = {t: 0 for t in TYPES}
90     for o in ops:
91         for t in o["dur"]:
92             busy[t] += o["dur"][t]
93     for ti, t in enumerate(TYPES):
94         M[ti, cj] = busy[t] / cmax * 100.0
95 fig, ax = plt.subplots(figsize=(6, 4))
96 im = ax.imshow(M, cmap="Blues", aspect="auto", vmin=0, vmax=100)
97 ax.set_xticks(range(len(cases))); ax.set_xticklabels([c[0] for c in cases])
98 ax.set_yticks(range(len(TYPES))); ax.set_yticklabels([lab(t) for t in TYPES
99 ])
100 for i in range(len(TYPES)):
101     for j in range(len(cases)):
102         ax.text(j, i, f"{M[i, j]:.0f}%", ha="center", va="center", fontsize
103 =8,
104 color="black" if M[i, j] < 60 else "white")
105 ax.set_title("设备利用率 (忙时/总工期)" if ZH else "Utilization (busy/
106 makespan)")
107 fig.colorbar(im, ax=ax, label="利用率 / %" if ZH else "util / %")
108 fig.tight_layout(); fig.savefig(path, dpi=150); plt.close(fig)
109 print("saved", path)
110
111 def buy_heatmap(path):
112     KP, KS = 5, 5
113     M = np.full((KP, KS), np.nan)
114     for kp in range(KP):
115         for ks in range(KS):
116             if kp * PRICE["polish"] + ks * PRICE["sensor"] > 500000:
117                 continue
118             cc = dict(POOL); cc["polish"] += kp; cc["sensor"] += ks
119             M[kp, ks] = solve(cc, ALL, ["G1", "G2"], time_limit=40)["cmax"] /

```

```

3600.0
117 fig, ax = plt.subplots(figsize=(6.4, 4.6))
118 cmap = plt.cm.viridis.copy(); cmap.set_bad("#cccccc")
119 im = ax.imshow(M, cmap=cmap, aspect="auto", origin="lower")
120 ax.set_xlabel("增购自动传感多功能机台数" if ZH else "Sensor added")
121 ax.set_ylabel("增购高速抛光机台数" if ZH else "Polisher added")
122 ax.set_xticks(range(KS)); ax.set_yticks(range(KP))
123 for kp in range(KP):
124     for ks in range(KS):
125         if not np.isnan(M[kp, ks]):
126             ax.text(ks, kp, f"{M[kp, ks]:.1f}", ha="center", va="center",
127                 fontsize=8,
128                 color="white" if M[kp, ks] > np.nanmin(M) + (np.nanmax(
129 M) - np.nanmin(M)) * 0.5 else "black")
130 ax.plot(3, 3, marker="*", color="red", markersize=18) # 最优方案 (+3,+3)
131 ax.set_title("购置组合对总工期的影响 / h (灰=超预算, 星=最优)" if ZH else "
132 Makespan vs purchase / h")
133 fig.colorbar(im, ax=ax, label="总工期 / h" if ZH else "makespan / h")
134 fig.tight_layout(); fig.savefig(path, dpi=150); plt.close(fig)
135 print("saved", path)
136
137 if __name__ == "__main__":
138     r2 = solve(G1, ALL, ["G1"], time_limit=120)
139     r3 = solve(P00L, ALL, ["G1", "G2"], time_limit=180)
140     cc = dict(P00L); cc["polish"] += 3; cc["sensor"] += 3
141     r4 = solve(cc, ALL, ["G1", "G2"], time_limit=60)
142     gantt(r2, "问题二 调度甘特图 (Cmax=144880 s)" if ZH else "Q2 Gantt", "figs/
143 gantt_q2.png")
144     gantt(r3, "问题三 调度甘特图 (Cmax=73575 s)" if ZH else "Q3 Gantt", "figs/
145 gantt_q3.png")
146     gantt(r4, "问题四 调度甘特图 (Cmax=32608 s)" if ZH else "Q4 Gantt", "figs/
147 gantt_q4.png")
148     load_heatmap("figs/load_heatmap.png")
149     util_heatmap("figs/util_heatmap.png")
150     buy_heatmap("figs/buy_heatmap.png")
151     print("ZH font:", ZH)

```

B 各问结果明细表

说明： 下列各表按设备类型分组给出最优调度。由可分负载假设，同一类型的全部机器成组作业、时间线一致；表中”设备编号”列出该类型参与作业的机器，多台机器在所列表段内同时作业。起始/结束时间为该类设备的作业区间，转运时间体现为相邻作业之间的空档（首道作业前的空档为由基地出发的初始转运）。

表 9 问题一结果（班组 1 整修 A 车间， $C_{\max} = 37280\text{ s}$ ）

序号	设备编号	起始时间	结束时间	持续 (s)	工序
1	精密灌装机 1-1~1-5	00:03:20	00:21:20	1080	A1
2	自动化输送臂 1-1~1-4	00:03:20	00:21:20	1080	A1
3	高速抛光机 1-1	00:21:20	05:21:20	18000	A2
4	工业清洗机 1-1~1-5	00:21:20	00:45:20	1440	A2
5	自动传感多功能机 1-1	05:21:20	10:21:20	18000	A3

表 10 问题二结果（班组 1 整修 A–E， $C_{\max} = 144880\text{ s}$ ）

序号	设备编号	起始时间	结束时间	持续 (s)	工序
自动化输送臂 1-1~1-4					
1	输送臂 1-1~1-4	00:03:50	00:47:02	2592	C1
2	输送臂 1-1~1-4	01:11:44	01:33:20	1296	C3
3	输送臂 1-1~1-4	01:37:40	02:17:40	2400	D2
4	输送臂 1-1~1-4	02:54:41	04:09:41	4500	B2
5	输送臂 1-1~1-4	14:15:00	14:36:36	1296	C3
6	输送臂 1-1~1-4	14:45:21	15:03:21	1080	A1
7	输送臂 1-1~1-4	26:32:00	26:53:36	1296	C3
工业清洗机 1-1~1-5					
8	清洗机 1-1~1-5	00:03:50	00:38:24	2074	C1
9	清洗机 1-1~1-5	00:42:44	01:11:32	1728	D1
10	清洗机 1-1~1-5	01:33:20	02:21:20	2880	C4

续表 10

序号	设备编号	起始时间	结束时间	持续 (s)	工序
11	清洗机 1-1~1-5	02:30:30	02:44:54	864	B1
12	清洗机 1-1~1-5	02:50:54	03:38:54	2880	E1
13	清洗机 1-1~1-5	14:22:06	15:34:06	4320	E3
14	清洗机 1-1~1-5	17:32:01	18:20:01	2880	C4
15	清洗机 1-1~1-5	21:00:46	21:24:46	1440	A2
16	清洗机 1-1~1-5	29:50:20	30:38:20	2880	C4
精密灌装机 1-1~1-5					
17	灌装机 1-1~1-5	00:47:02	01:11:44	1482	C2
18	灌装机 1-1~1-5	01:11:44	01:33:20	1296	C3
19	灌装机 1-1~1-5	01:37:40	02:25:40	2880	D2
20	灌装机 1-1~1-5	02:25:40	02:41:06	926	D3
21	灌装机 1-1~1-5	02:54:41	04:24:41	5400	B2
22	灌装机 1-1~1-5	04:24:41	04:37:02	741	B3
23	灌装机 1-1~1-5	05:51:46	06:12:21	1235	E2
24	灌装机 1-1~1-5	14:15:00	14:36:36	1296	C3
25	灌装机 1-1~1-5	14:45:21	15:03:21	1080	A1
26	灌装机 1-1~1-5	26:32:00	26:53:36	1296	C3
自动传感多功能机 1-1					
27	传感机 1-1	04:57:40	09:57:40	18000	D4
28	传感机 1-1	10:15:00	14:15:00	14400	C5
29	传感机 1-1	14:22:06	16:22:06	7200	E3
30	传感机 1-1	17:27:40	22:27:40	18000	D5
31	传感机 1-1	22:32:00	26:32:00	14400	C5
32	传感机 1-1	26:41:10	30:17:10	12960	B4
33	传感机 1-1	30:25:40	35:25:40	18000	A3
34	传感机 1-1	35:34:25	39:34:25	14400	C5

续表 10

序号	设备编号	起始时间	结束时间	持续 (s)	工序
高速抛光机 1-1					
35	抛光机 1-1	01:33:20	04:53:20	12000	C4
36	抛光机 1-1	04:57:40	17:27:40	45000	D4
37	抛光机 1-1	17:32:01	20:52:01	12000	C4
38	抛光机 1-1	21:00:46	26:00:46	18000	A2
39	抛光机 1-1	26:41:10	29:41:10	10800	B4
40	抛光机 1-1	29:50:20	33:10:20	12000	C4
41	抛光机 1-1	33:14:40	40:14:40	25200	D6

表 11 问题三结果（班组 1、2 整修 A-E, $C_{\max} = 73575\text{ s}$ ）

序号	设备编号	起始时间	结束时间	持续 (s)	工序	班组
自动化输送臂 1-1~1-4, 2-1~2-4						
1	输送臂池 (8 台)	00:05:10	00:26:46	1296	C1	1,2
2	输送臂池 (8 台)	00:39:07	00:49:55	648	C3	1,2
3	输送臂池 (8 台)	00:54:15	01:14:15	1200	D2	1,2
4	输送臂池 (8 台)	02:18:52	02:56:22	2250	B2	1,2
5	输送臂池 (8 台)	08:19:55	08:30:43	648	C3	1,2
6	输送臂池 (8 台)	08:54:34	09:03:34	540	A1	1,2
7	输送臂池 (8 台)	13:23:35	13:34:23	648	C3	1,2
工业清洗机 1-1~1-5, 2-1~2-5						
8	清洗机池 (10 台)	00:05:10	00:22:27	1037	C1	1,2
9	清洗机池 (10 台)	00:26:47	00:41:11	864	D1	1,2
10	清洗机池 (10 台)	00:49:55	01:13:55	1440	C4	1,2
11	清洗机池 (10 台)	01:23:05	01:30:17	432	B1	1,2
12	清洗机池 (10 台)	01:36:17	02:00:17	1440	E1	1,2
13	清洗机池 (10 台)	05:12:50	05:48:50	2160	E3	1,2

续表 11

序号	设备编号	起始时间	结束时间	持续 (s)	工序	班组
14	清洗机池 (10 台)	08:53:35	09:17:35	1440	C4	1,2
15	清洗机池 (10 台)	10:42:20	10:54:20	720	A2	1,2
16	清洗机池 (10 台)	15:11:55	15:35:55	1440	C4	1,2
精密灌装机 1-1~1-5, 2-1~2-5						
17	灌装机池 (10 台)	00:26:46	00:39:07	741	C2	1,2
18	灌装机池 (10 台)	00:39:07	00:49:55	648	C3	1,2
19	灌装机池 (10 台)	00:54:15	01:18:15	1440	D2	1,2
20	灌装机池 (10 台)	01:18:15	01:25:58	463	D3	1,2
21	灌装机池 (10 台)	02:00:17	02:10:35	618	E2	1,2
22	灌装机池 (10 台)	02:18:52	03:03:52	2700	B2	1,2
23	灌装机池 (10 台)	08:19:55	08:30:43	648	C3	1,2
24	灌装机池 (10 台)	08:39:53	08:46:04	371	B3	1,2
25	灌装机池 (10 台)	08:54:34	09:03:34	540	A1	1,2
26	灌装机池 (10 台)	13:23:35	13:34:23	648	C3	1,2
自动传感多功能机 1-1, 2-1						
27	传感机池 (2 台)	02:34:15	05:04:15	9000	D4	1,2
28	传感机池 (2 台)	05:12:50	06:12:50	3600	E3	1,2
29	传感机池 (2 台)	06:19:55	08:19:55	7200	C5	1,2
30	传感机池 (2 台)	08:49:15	11:19:15	9000	D5	1,2
31	传感机池 (2 台)	11:23:35	13:23:35	7200	C5	1,2
32	传感机池 (2 台)	13:32:45	15:20:45	6480	B4	1,2
33	传感机池 (2 台)	15:29:15	17:59:15	9000	A3	1,2
34	传感机池 (2 台)	18:08:00	20:08:00	7200	C5	1,2
高速抛光机 1-1, 2-1						
35	抛光机池 (2 台)	00:49:55	02:29:55	6000	C4	1,2
36	抛光机池 (2 台)	02:34:15	08:49:15	22500	D4	1,2

续表 11

序号	设备编号	起始时间	结束时间	持续 (s)	工序	班组
37	抛光机池 (2 台)	08:53:35	10:33:35	6000	C4	1,2
38	抛光机池 (2 台)	10:42:20	13:12:20	9000	A2	1,2
39	抛光机池 (2 台)	13:32:45	15:02:45	5400	B4	1,2
40	抛光机池 (2 台)	15:11:55	16:51:55	6000	C4	1,2
41	抛光机池 (2 台)	16:56:15	20:26:15	12600	D6	1,2

表 12 问题四结果 (购置后整修 A-E, $C_{\max} = 32608\text{ s}$)

序号	设备编号	起始时间	结束时间	持续 (s)	工序	班组
自动化输送臂 1-1~1-4, 2-1~2-4 (8 台, 未增购)						
1	输送臂池 (8 台)	00:05:10	00:26:46	1296	C1	1,2
2	输送臂池 (8 台)	00:39:07	00:49:55	648	C3	1,2
3	输送臂池 (8 台)	00:54:15	01:14:15	1200	D2	1,2
4	输送臂池 (8 台)	03:01:38	03:39:08	2250	B2	1,2
5	输送臂池 (8 台)	04:01:55	04:12:43	648	C3	1,2
6	输送臂池 (8 台)	04:21:28	04:30:28	540	A1	1,2
7	输送臂池 (8 台)	05:58:35	06:09:23	648	C3	1,2
工业清洗机 1-1~1-5, 2-1~2-5 (10 台, 未增购)						
8	清洗机池 (10 台)	00:05:10	00:22:27	1037	C1	1,2
9	清洗机池 (10 台)	00:26:47	00:41:11	864	D1	1,2
10	清洗机池 (10 台)	00:49:55	01:13:55	1440	C4	1,2
11	清洗机池 (10 台)	01:43:39	02:07:39	1440	E1	1,2
12	清洗机池 (10 台)	02:13:39	02:20:51	432	B1	1,2
13	清洗机池 (10 台)	02:42:50	03:18:50	2160	E3	1,2
14	清洗机池 (10 台)	04:12:43	04:36:43	1440	C4	1,2
15	清洗机池 (10 台)	05:01:28	05:13:28	720	A2	1,2

续表 12

序号	设备编号	起始时间	结束时间	持续 (s)	工序	班组
16	清洗机池 (10 台)	06:55:08	07:19:08	1440	C4	1,2
精密灌装机 1-1~1-5, 2-1~2-5 (10 台, 未增购)						
17	灌装机池 (10 台)	00:26:46	00:39:07	741	C2	1,2
18	灌装机池 (10 台)	00:39:07	00:49:55	648	C3	1,2
19	灌装机池 (10 台)	00:54:15	01:18:15	1440	D2	1,2
20	灌装机池 (10 台)	01:18:15	01:25:58	463	D3	1,2
21	灌装机池 (10 台)	02:07:39	02:17:57	618	E2	1,2
22	灌装机池 (10 台)	03:01:38	03:46:38	2700	B2	1,2
23	灌装机池 (10 台)	04:01:55	04:12:43	648	C3	1,2
24	灌装机池 (10 台)	04:21:28	04:30:28	540	A1	1,2
25	灌装机池 (10 台)	04:38:58	04:45:09	371	B3	1,2
26	灌装机池 (10 台)	05:58:35	06:09:23	648	C3	1,2
自动传感多功能机 1-1, 2-1 + 增购 3 台 (共 5 台)						
27	传感机池 (5 台)	01:34:15	02:34:15	3600	D4	1,2
28	传感机池 (5 台)	02:42:50	03:06:50	1440	E3	1,2
29	传感机池 (5 台)	03:13:55	04:01:55	2880	C5	1,2
30	传感机池 (5 台)	04:06:15	05:06:15	3600	D5	1,2
31	传感机池 (5 台)	05:10:35	05:58:35	2880	C5	1,2
32	传感机池 (5 台)	06:09:58	06:53:10	2592	B4	1,2
33	传感机池 (5 台)	07:01:40	08:01:40	3600	A3	1,2
34	传感机池 (5 台)	08:10:25	08:58:25	2880	C5	1,2
高速抛光机 1-1, 2-1 + 增购 3 台 (共 5 台)						
35	抛光机池 (5 台)	00:49:55	01:29:55	2400	C4	1,2
36	抛光机池 (5 台)	01:34:15	04:04:15	9000	D4	1,2
37	抛光机池 (5 台)	04:12:43	04:52:43	2400	C4	1,2
38	抛光机池 (5 台)	05:01:28	06:01:28	3600	A2	1,2

续表 12

序号	设备编号	起始时间	结束时间	持续 (s)	工序	班组
39	抛光机池 (5 台)	06:09:58	06:45:58	2160	B4	1,2
40	抛光机池 (5 台)	06:55:08	07:35:08	2400	C4	1,2
41	抛光机池 (5 台)	07:39:28	09:03:28	5040	D6	1,2

表 13 问题四设备购买情况

设备名称	班组 1 购买台数	班组 2 购买台数	小计
自动化输送臂	0	0	0
工业清洗机	0	0	0
精密灌装机	0	0	0
自动传感多功能机	2	1	3
高速抛光机	2	1	3
购买设备总费用： $3 \times 80000 + 3 \times 75000 = 465000$ 元			